

"THIS IS THE EMERGENCY OVERRIDE
SYSTEM, WHICH CAN BE USED TO
REGAIN CONTROL OF THE AIRCRAFT.

COMPLETE INSTRUCTIONS FOR
ACTIVATING THIS SYSTEM ARE
AVAILABLE AS A GNU INFO PAGE."



<https://xkcd.com/912/>

Randall Munroe, CC BY-NC 2.5

Control Systems

Dan Boschen

September 2026

<https://dsp-coach.com>





Content is derived from courses on DSP and Python taught by The DSP Coach, LLC

Copyright © 2026 The DSP Coach, LLC

This presentation "Control Systems" by The DSP Coach is licensed under CC BY-NC-ND 4.0.

To view a copy of this license, visit <https://creativecommons.org/licenses/by-nc-nd/4.0/>

While every precaution has been taken in the preparation of this presentation, the author, publisher, and distribution partners assume no responsibility for any errors or omissions, or any damages resulting from the use of any information contained within it.

Suggested attribution:

"Control Systems," Dan Boschen, The DSP Coach, LLC, presented at the GNU Radio Conference 2026. <https://github.com/dsp-coach/grcon2026_control>.

Licensed under [CC BY-NC-ND 4.0](<https://creativecommons.org/licenses/by-nc-nd/4.0/>).

Outline



Relevant examples

Fundamental background

Analog control systems

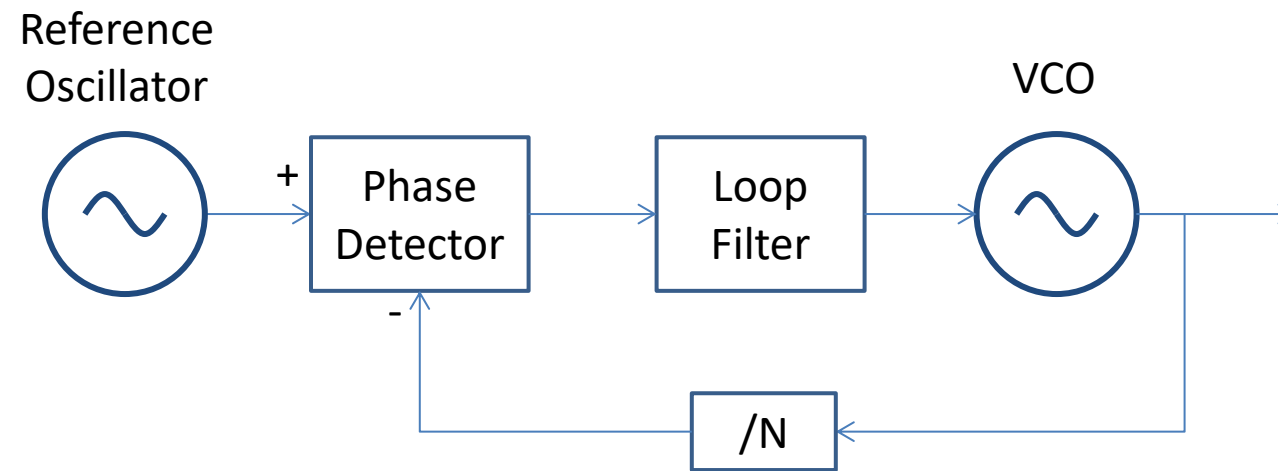
Mapping to digital and the z-transform

Digital control systems

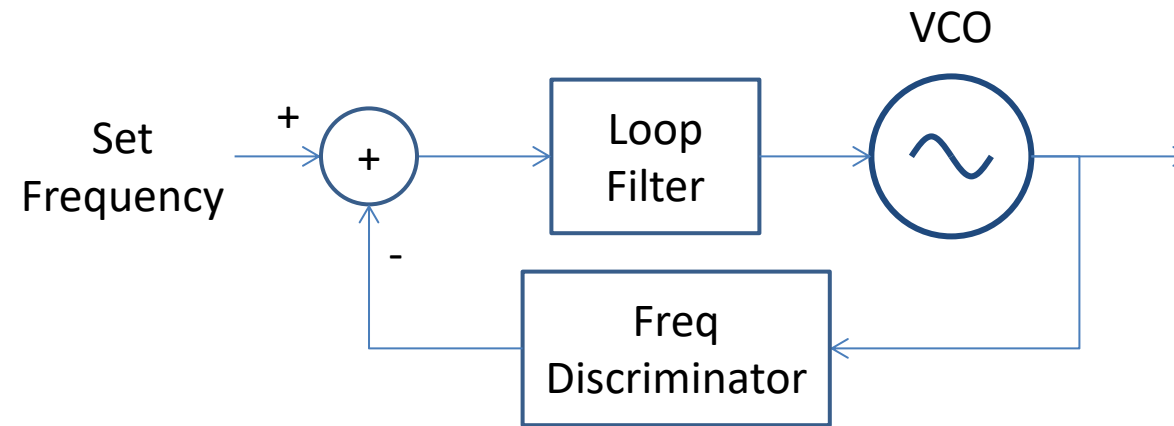
Using python for control system modelling



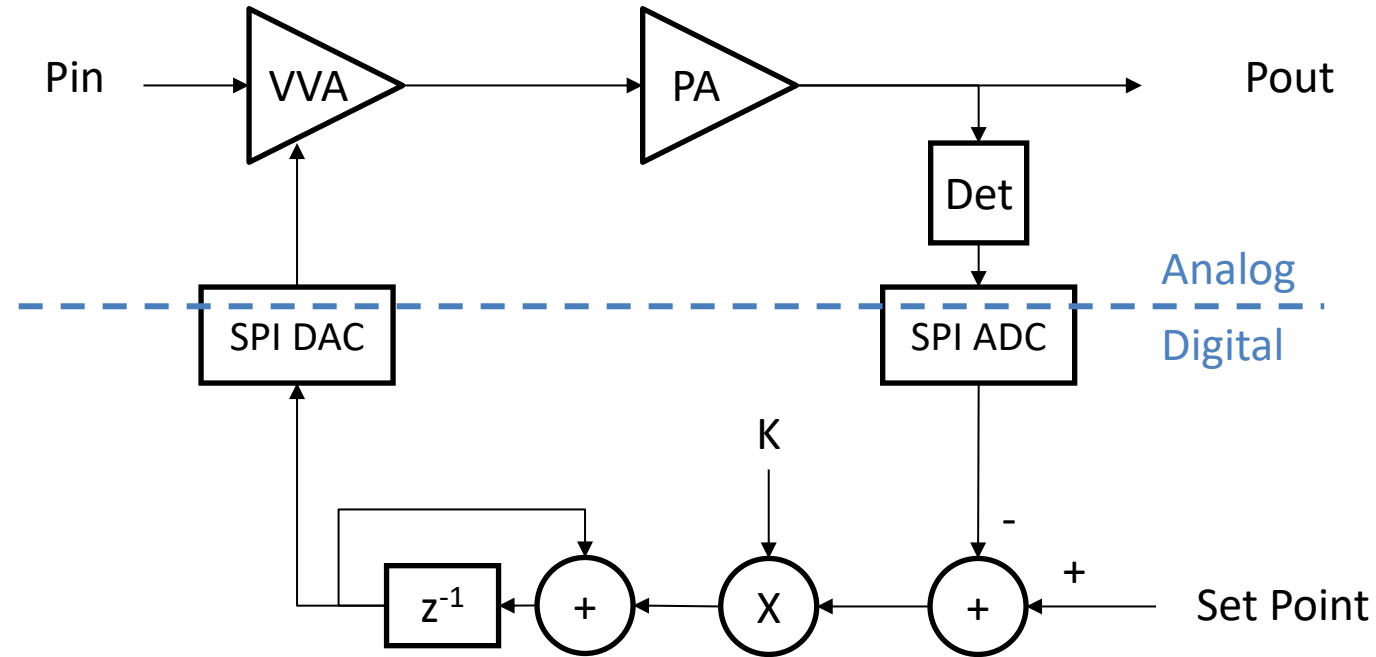
EXAMPLES



Integer-N Phase Lock Loop



Frequency Lock Loop

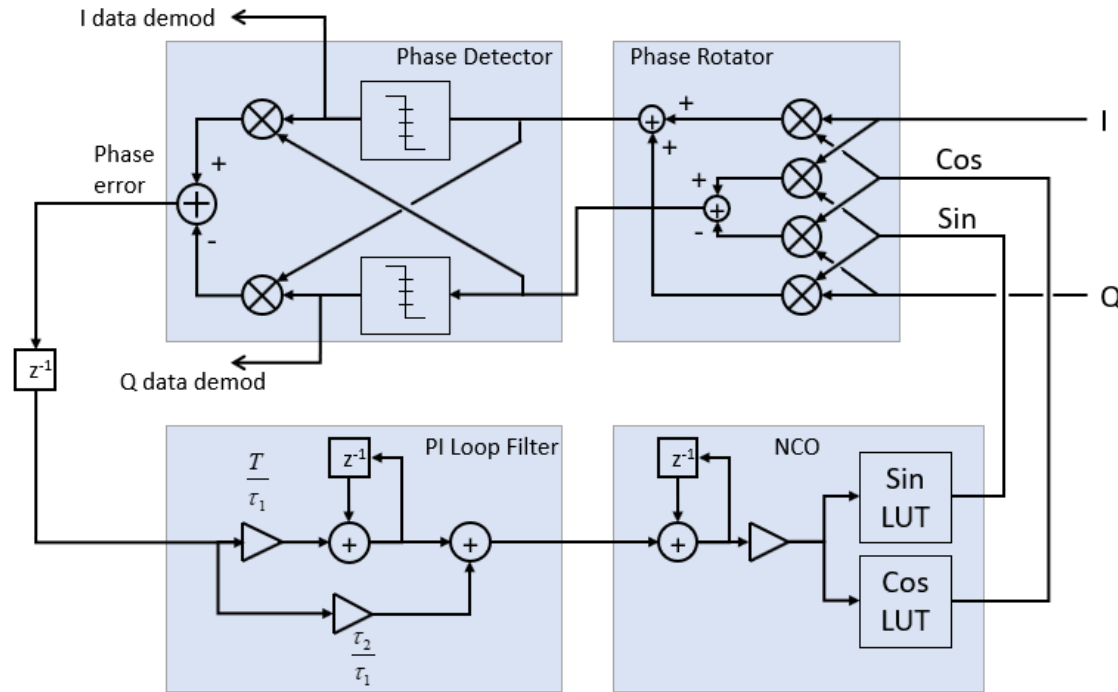


Mixed Signal Power Control Loop

Motivating Application



Reduce This:



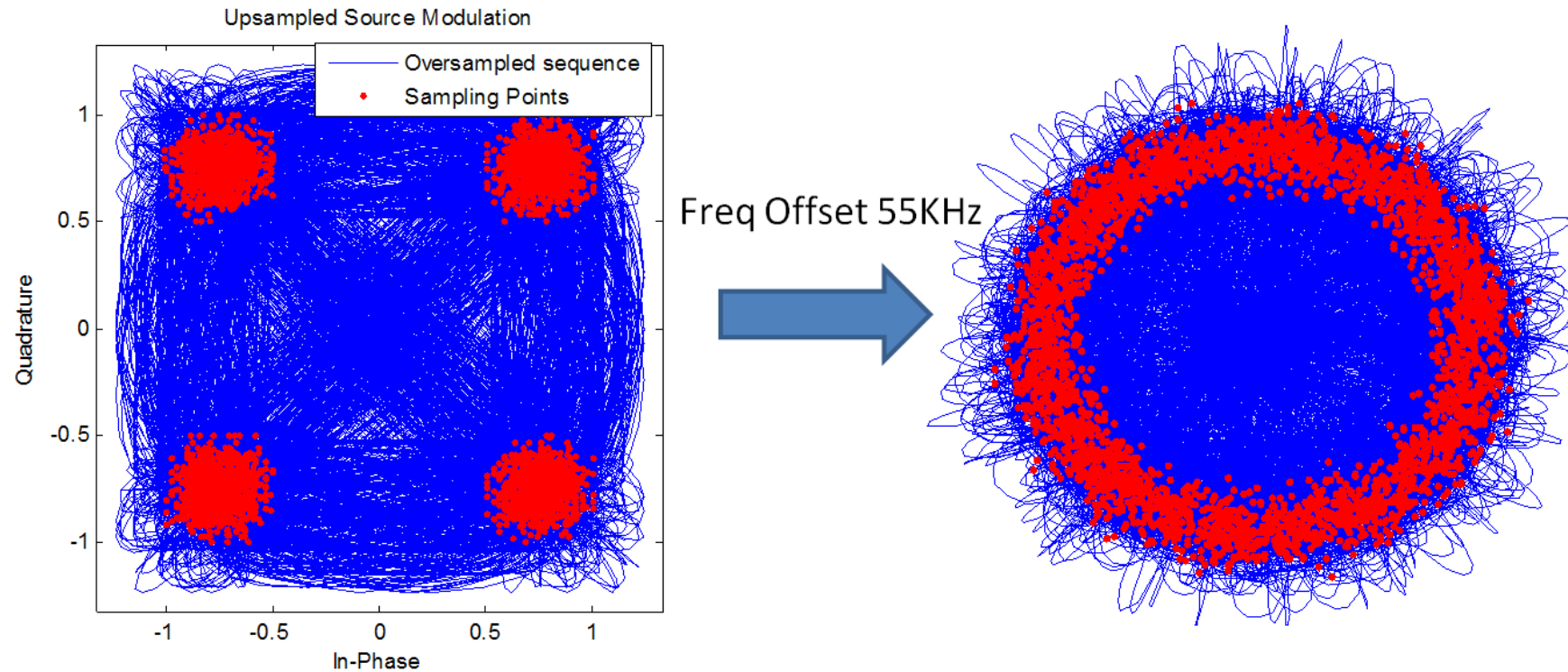
Decision Directed Carrier Recovery

To This:

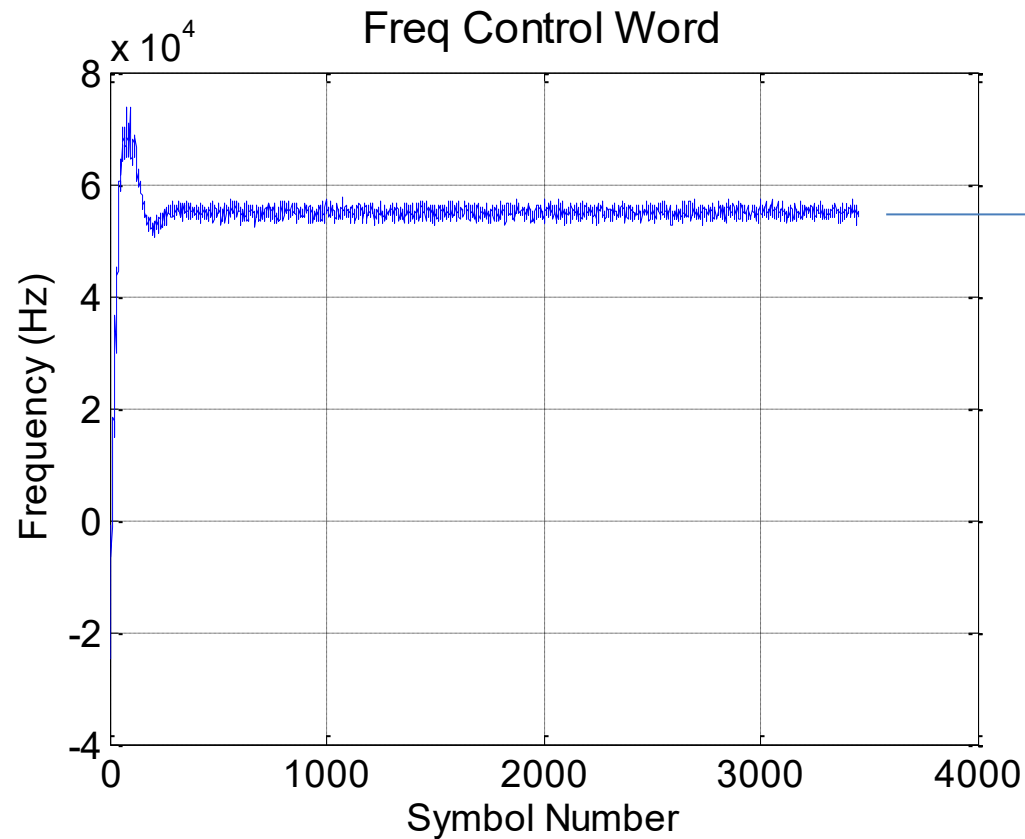
$$G_{OL}(z) = 0.0324 \frac{(z - 0.9695)}{(z - 1)^2}$$

Further details of getting to $G_{OL}(z)$ are in the "DSP for Software Radio" Course.
Bottom line shown here as a motivating intro.

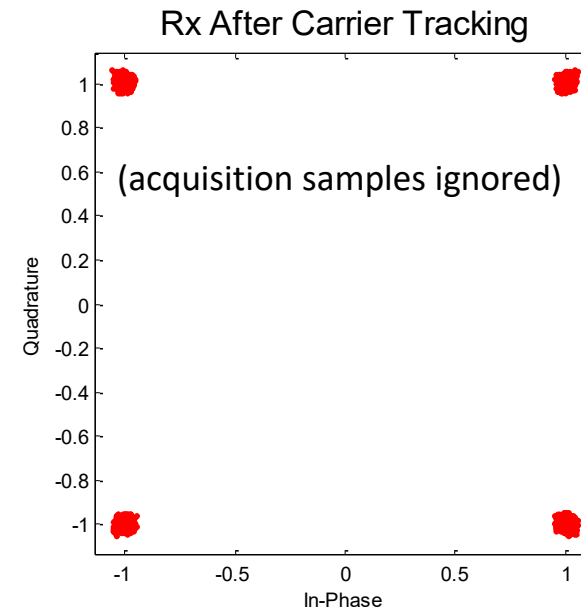
Example Application



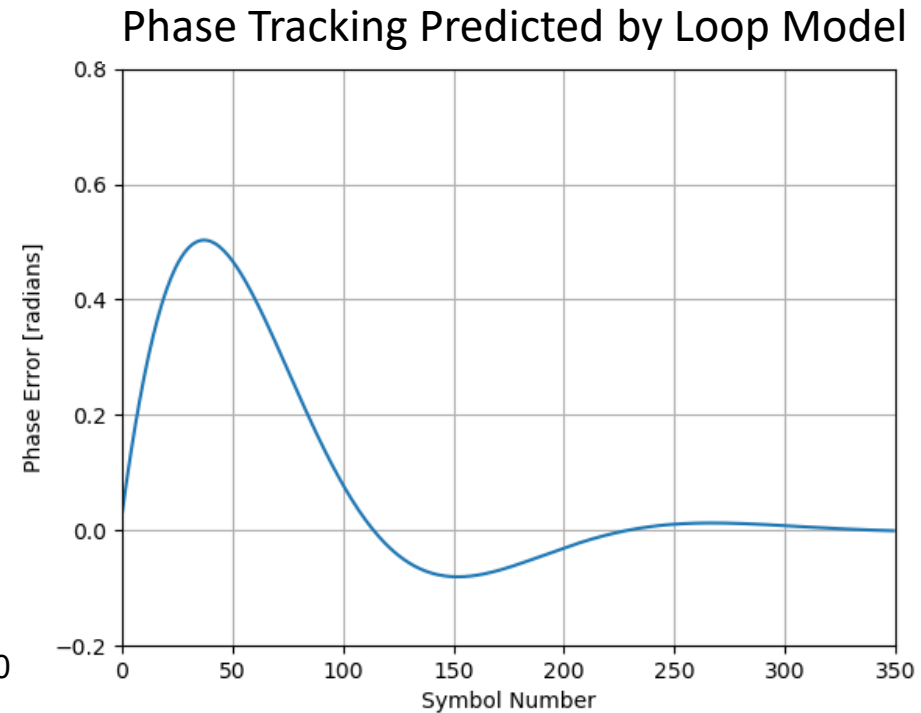
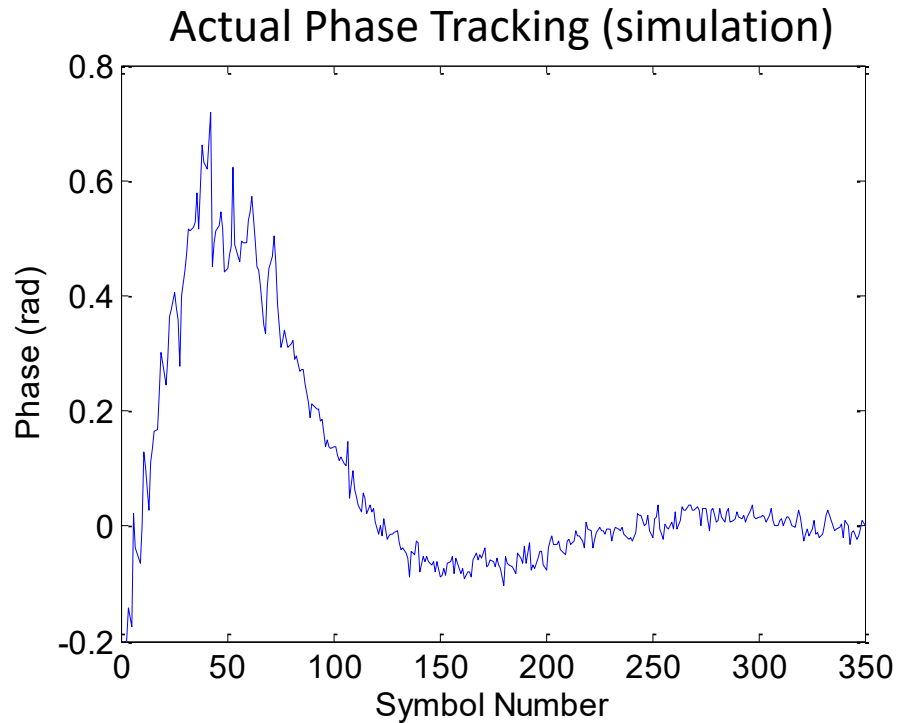
Example Application



Tracked to 55KHz
Frequency Offset



Example Application



$$G_{OL}(z) = 0.0324 \frac{(z - 0.9695)}{(z - 1)^2}$$

```
gol = con.tf([0.0324, -0.03142], [1, -2, 1])  
mag = 2*np.pi*55e3/12e6  
int = con.tf([1, 0], [1, -1], T)  
t, yout = con.step_response(mag*int/(1+gol), T = np.arange(350)*T)
```



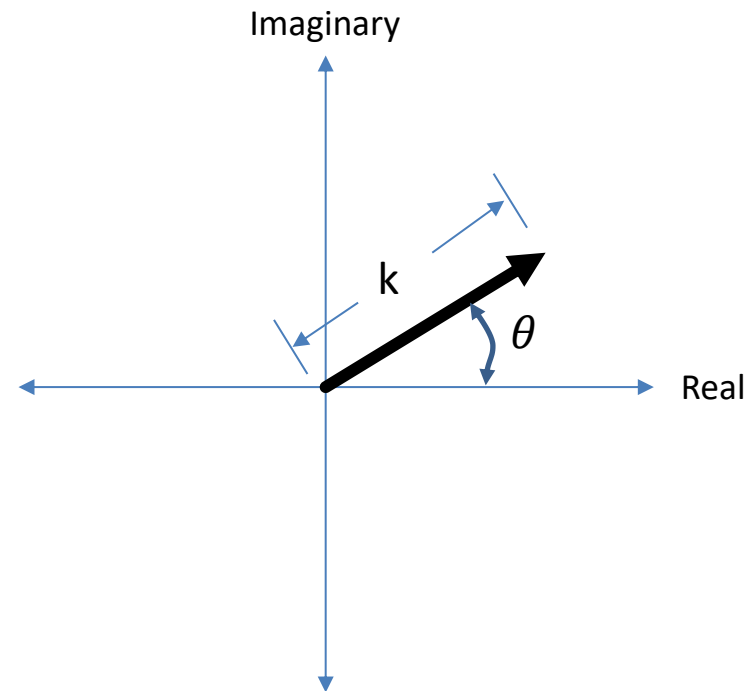


FUNDAMENTALS

Complex Representation



$$k e^{j\theta}$$



Example Complex Signals



$$k(t)e^{j\theta}$$

Magnitude changes with time

$$ke^{j\theta(t)}$$

Phase changes with time

$$k(t)e^{j\theta(t)}$$

Magnitude and Phase changes with time

$$K(\omega)e^{j\Theta(\omega)}$$

Magnitude and Phase changes with frequency

Euler's Formula

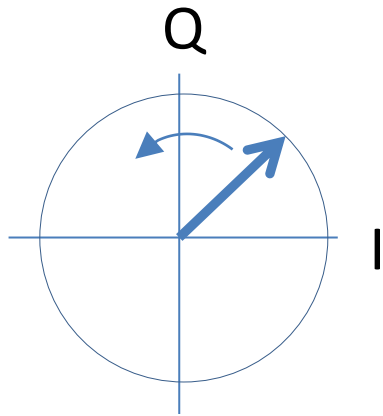


$$e^{j\phi} = \cos \phi + j \sin \phi$$
$$= I + jQ$$

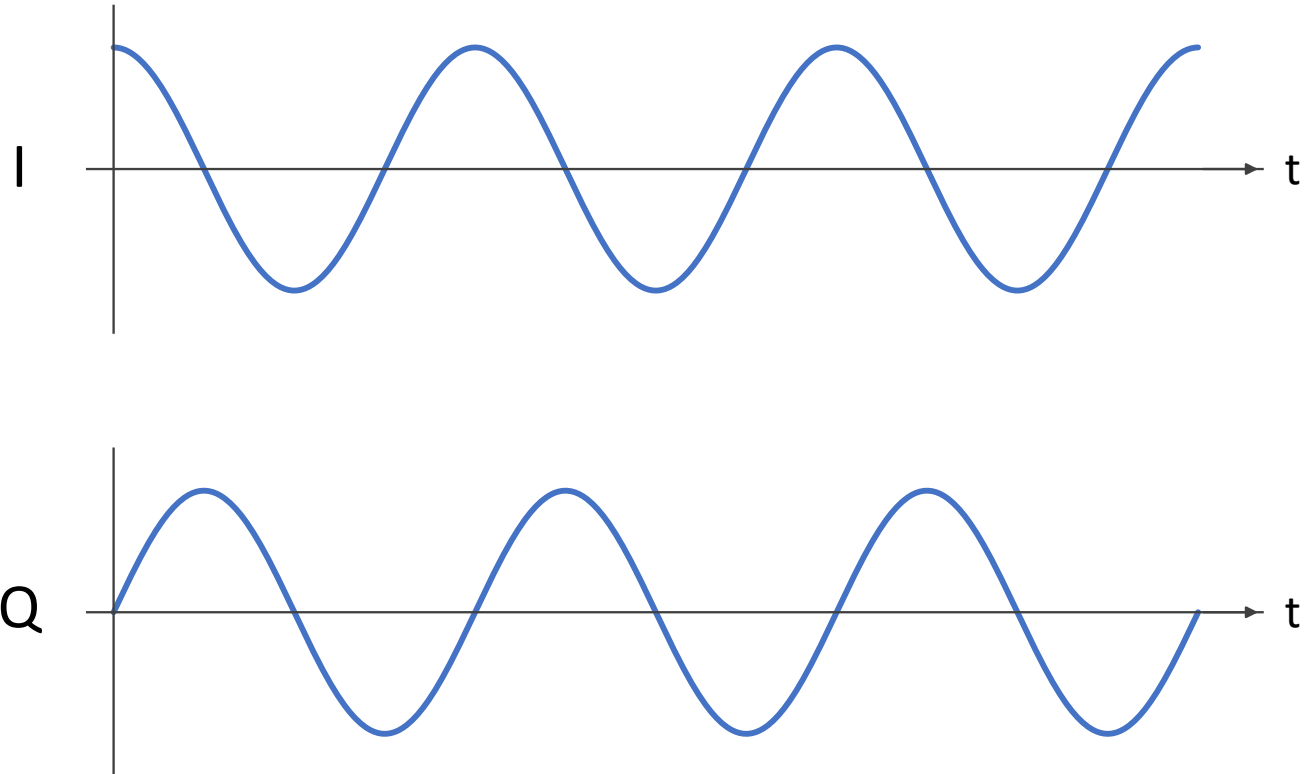
$$e^{j\phi} + e^{-j\phi} = 2 \cos \phi$$
$$e^{j\phi} - e^{-j\phi} = 2j \sin \phi$$



The Spinning Phasor



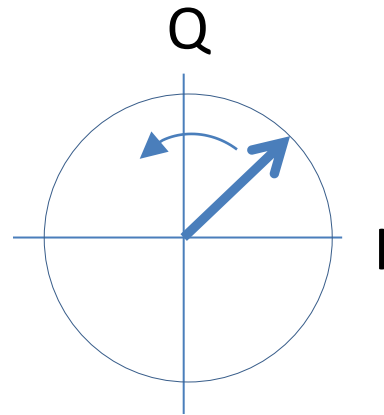
$$e^{+j2\pi f_o t}$$



Complex Signals in Time & Freq

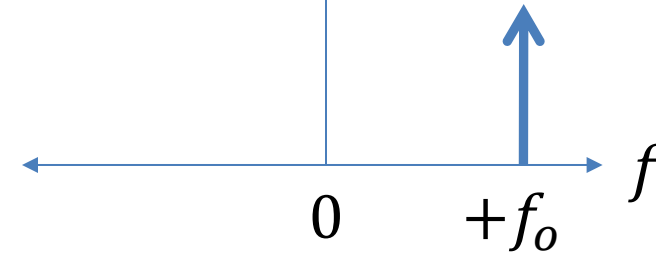


Time Domain



$$e^{+j2\pi f_o t}$$

Frequency Domain

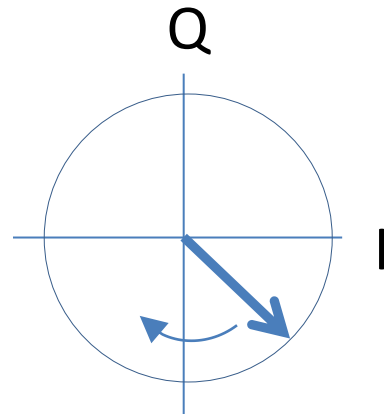


$$\delta(f - f_o)$$

Complex Signals in Time & Freq

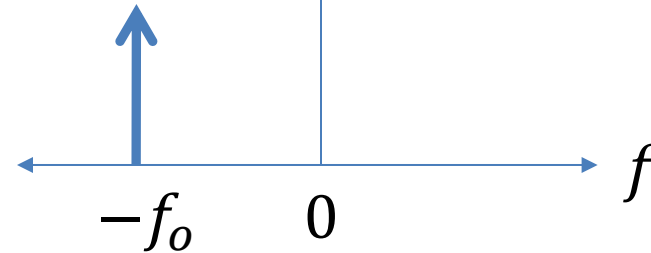


Time Domain



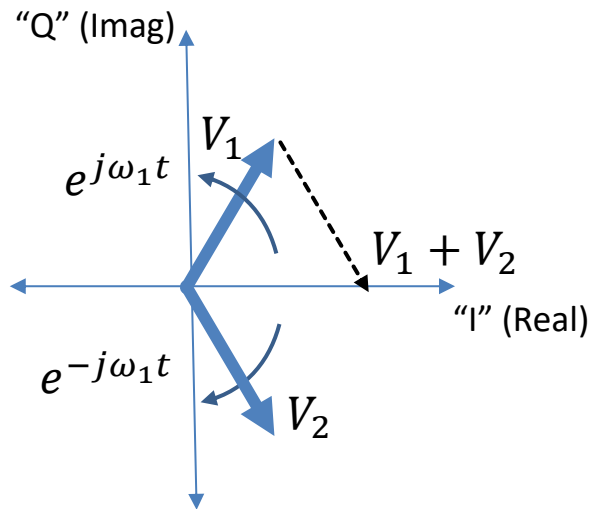
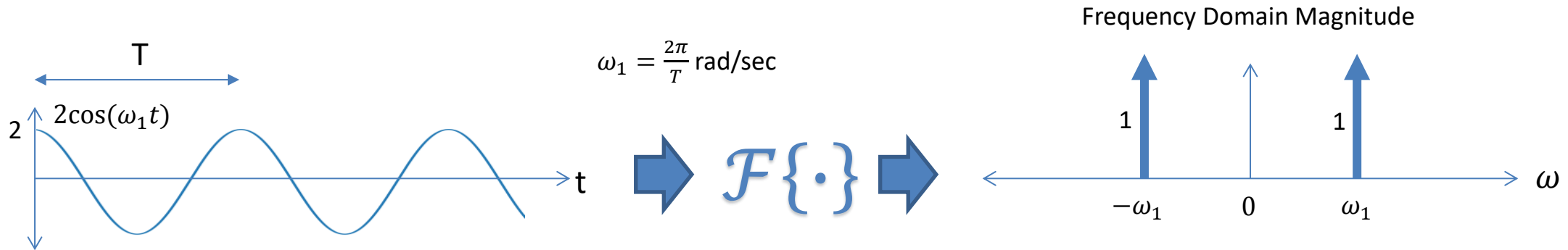
$$e^{-j2\pi f_o t}$$

Frequency Domain



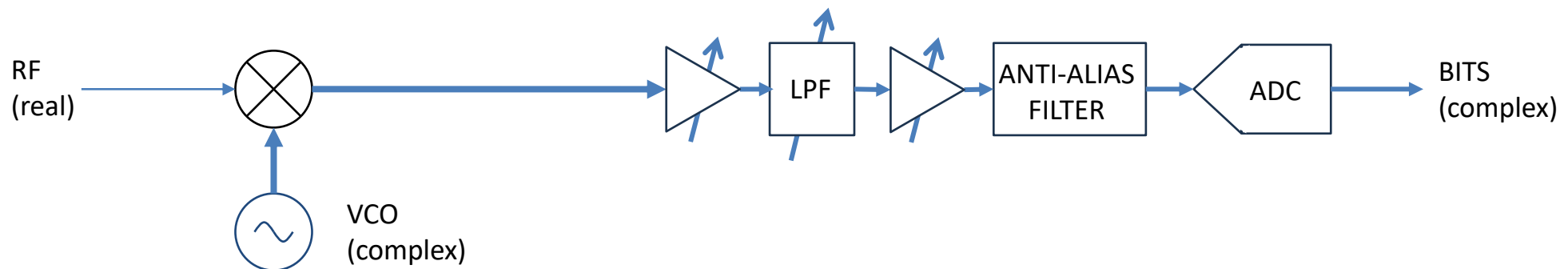
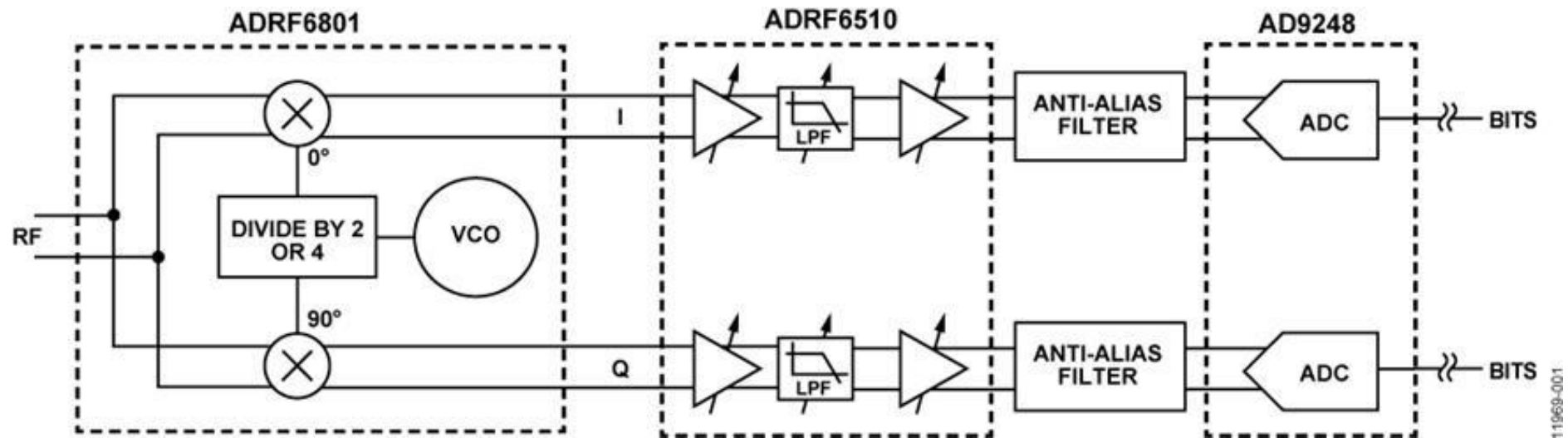
$$\delta(f + f_o)$$

Positive and Negative Frequencies



$$2\cos(\omega_1 t) = e^{j\omega_1 t} + e^{-j\omega_1 t}$$

Do Complex Signals "Exist" in the "Real" World?

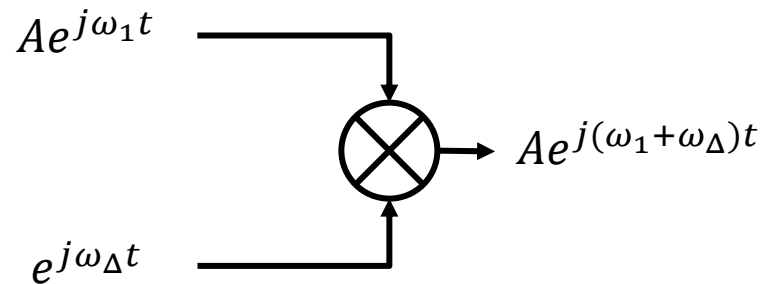


A Motivation for Using $ke^{j\theta}$

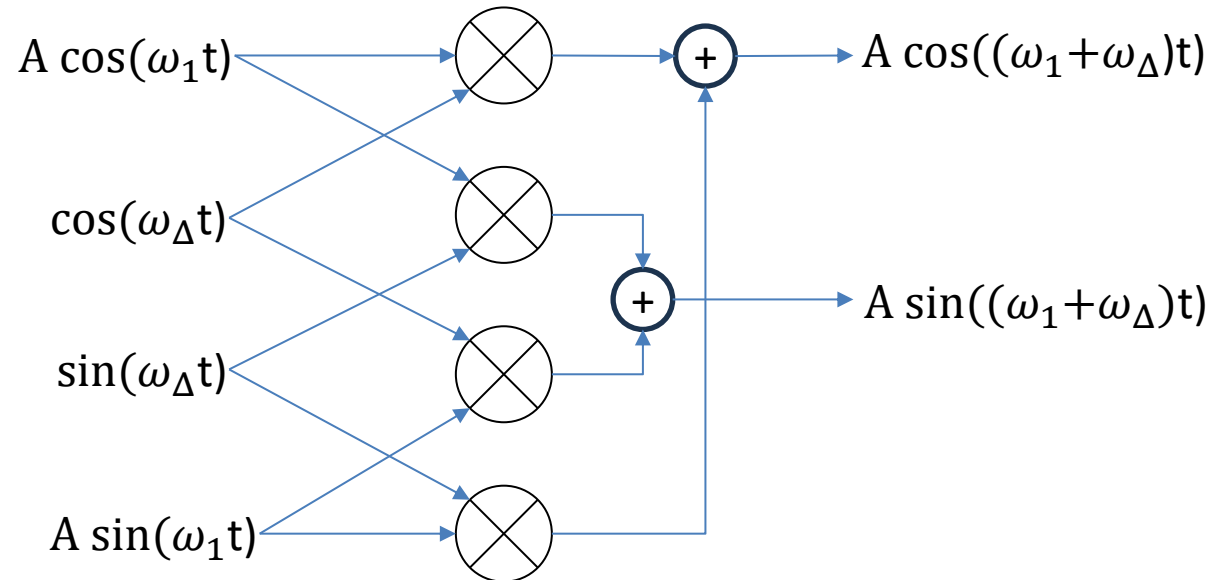


Try explaining how a full complex multiplier can be used to shift frequency...

Using Exponentials

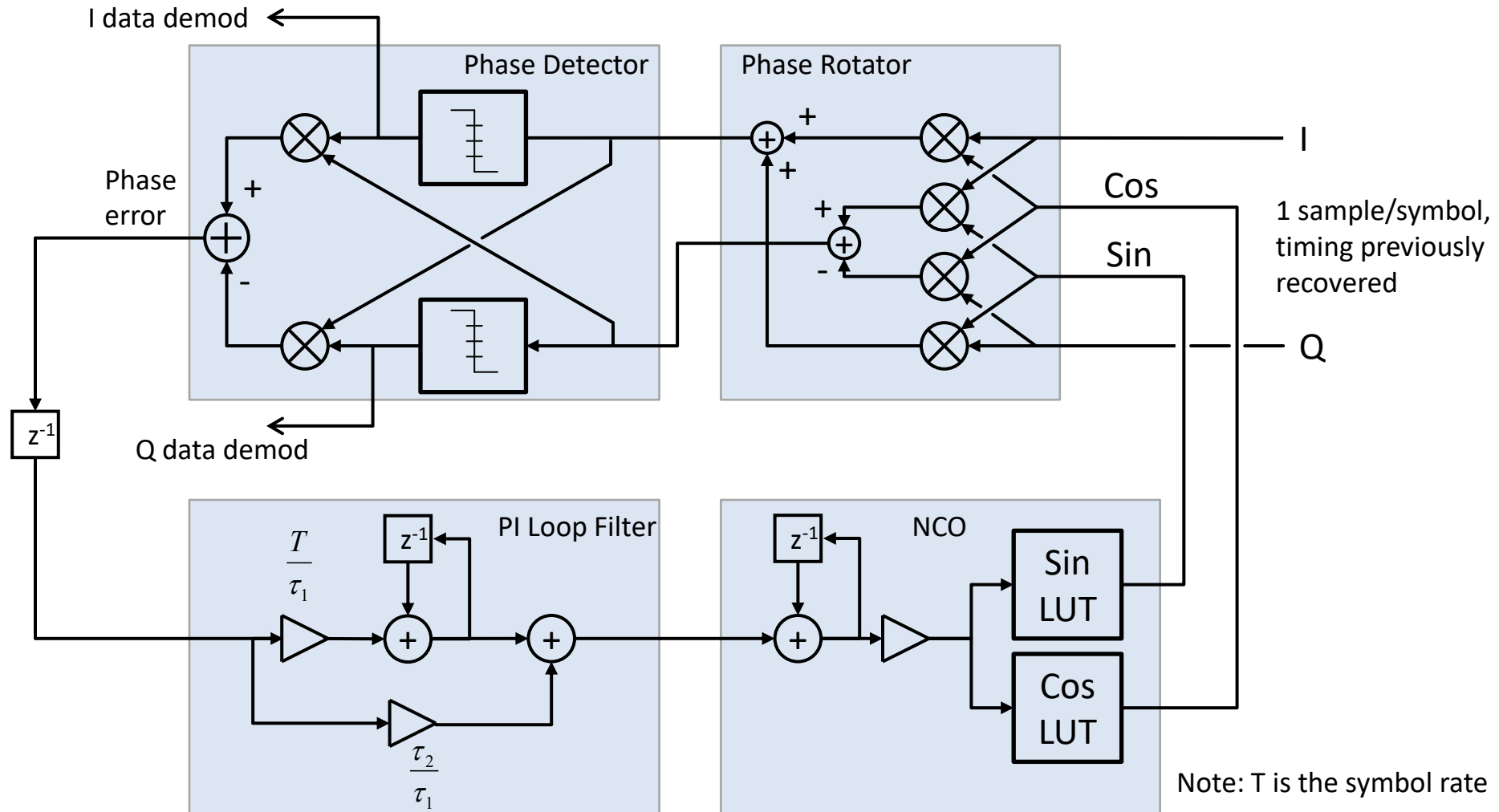


Using Sines and Cosines

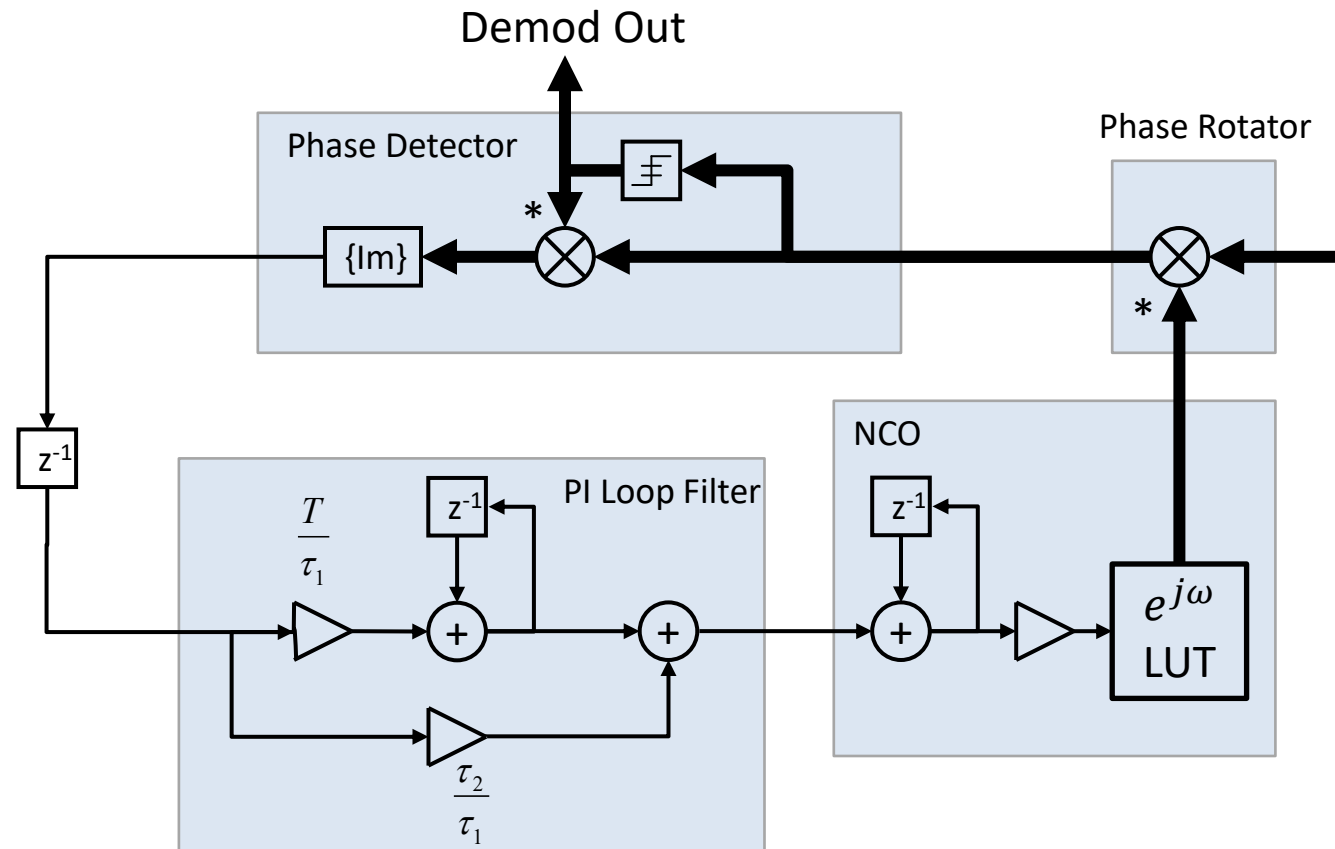




Decision Directed CR Loop



Decision Directed CR Loop



Note: T is the symbol rate



FOURIER AND LAPLACE TRA

Fourier and Laplace



$$X(\omega) \equiv \int_{t=-\infty}^{\infty} x(t)e^{-j\omega t} dt$$



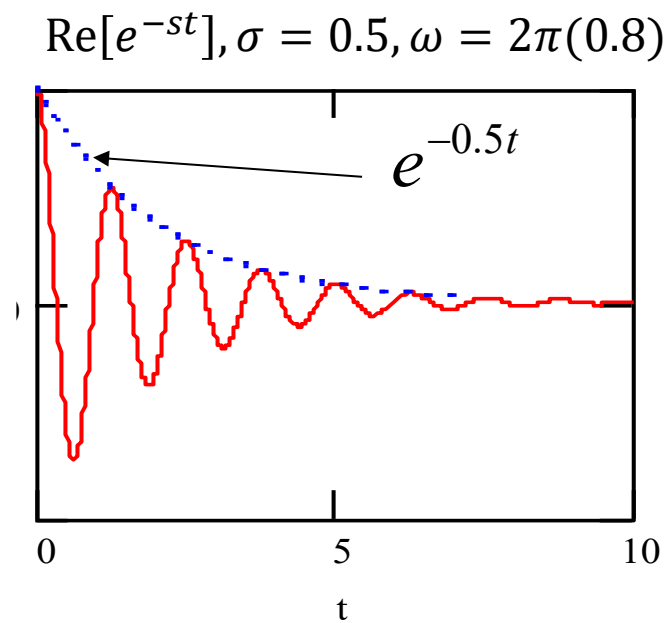
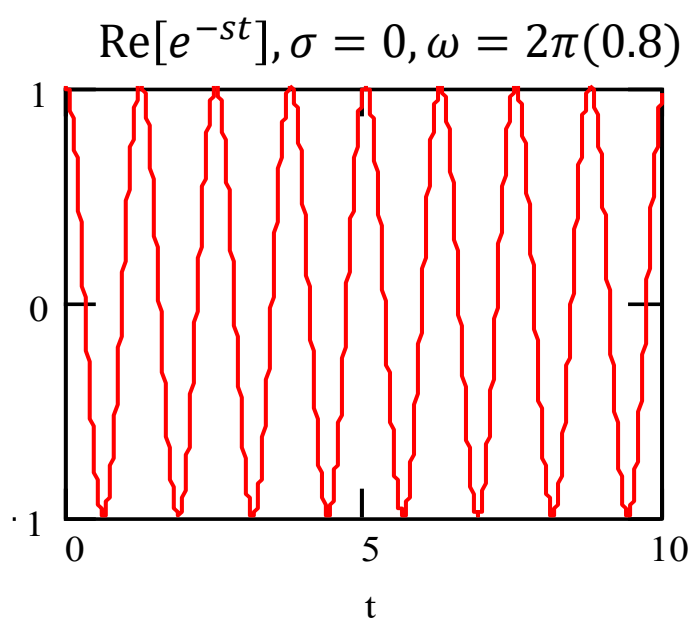
$$X(s) \equiv \int_{t=0}^{\infty} x(t)e^{-st} dt$$

$$e^{st}$$



s is a complex variable: $s = \sigma + j\omega$

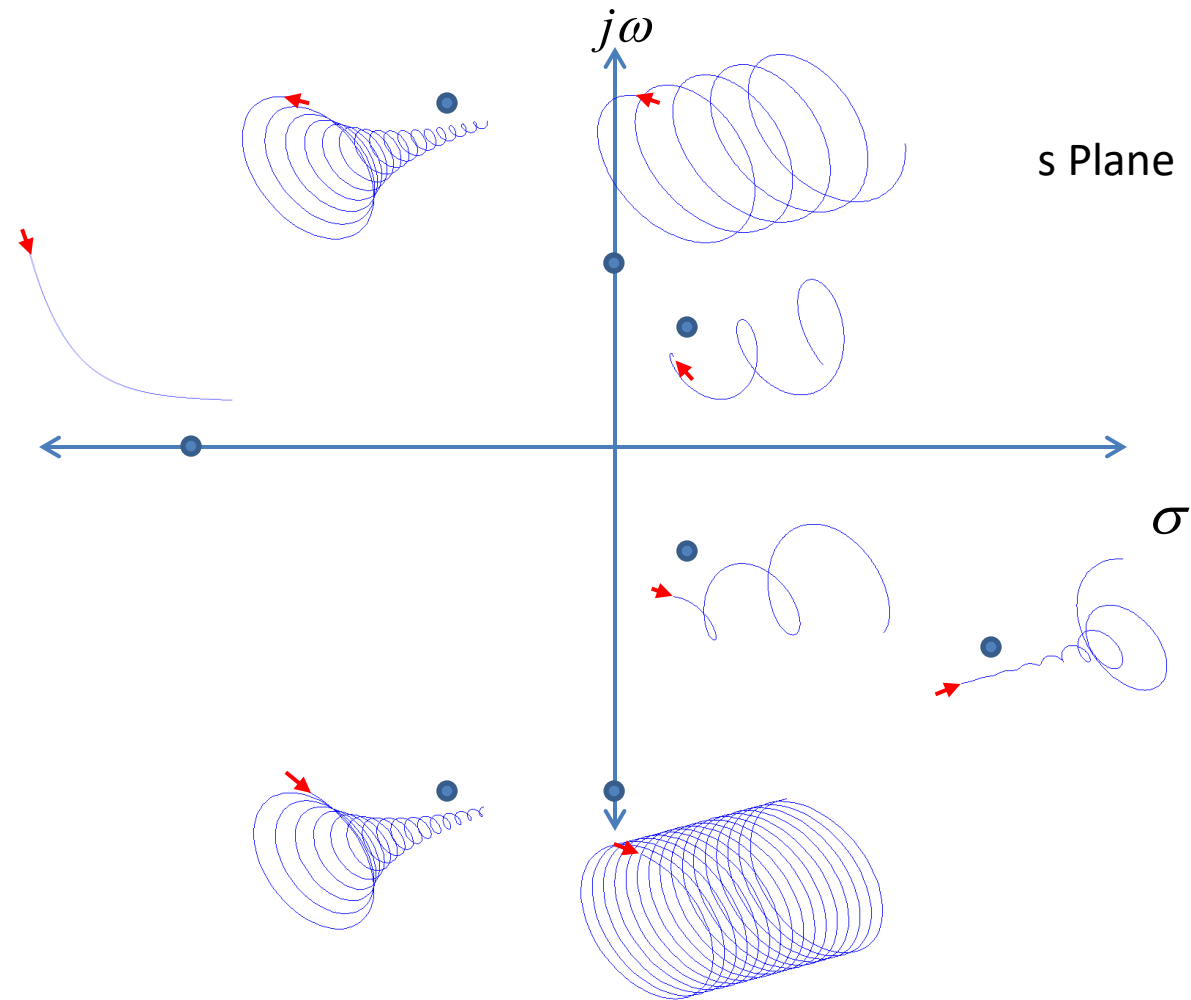
$$e^{-st} = e^{-(\sigma + j\omega)t} = e^{-\sigma t} e^{-j\omega t}$$





THE S AND Z PLANES

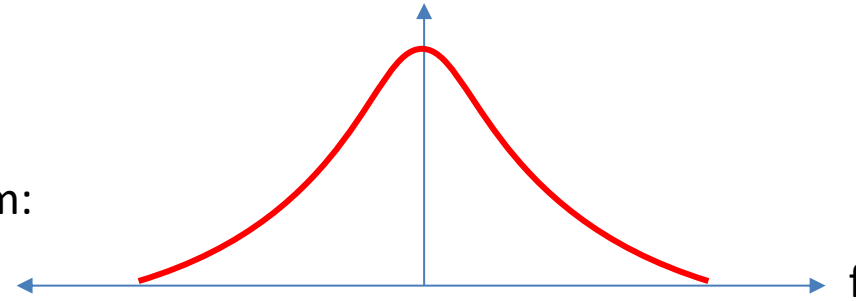
e^{st} for all values of s



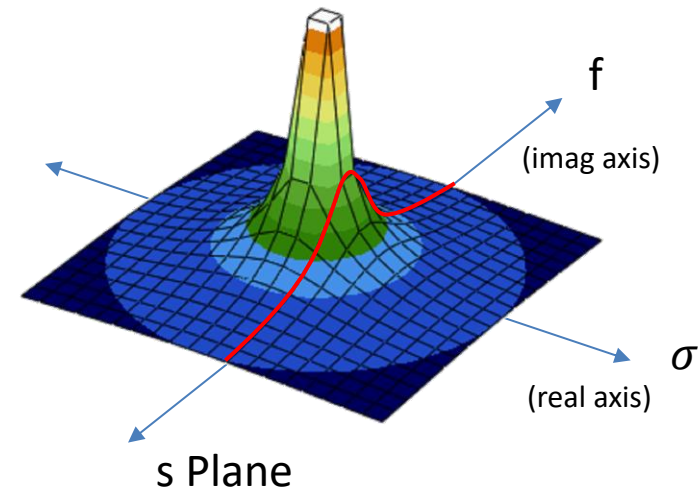
The s-plane



Graphing Magnitude of the Fourier Transform:



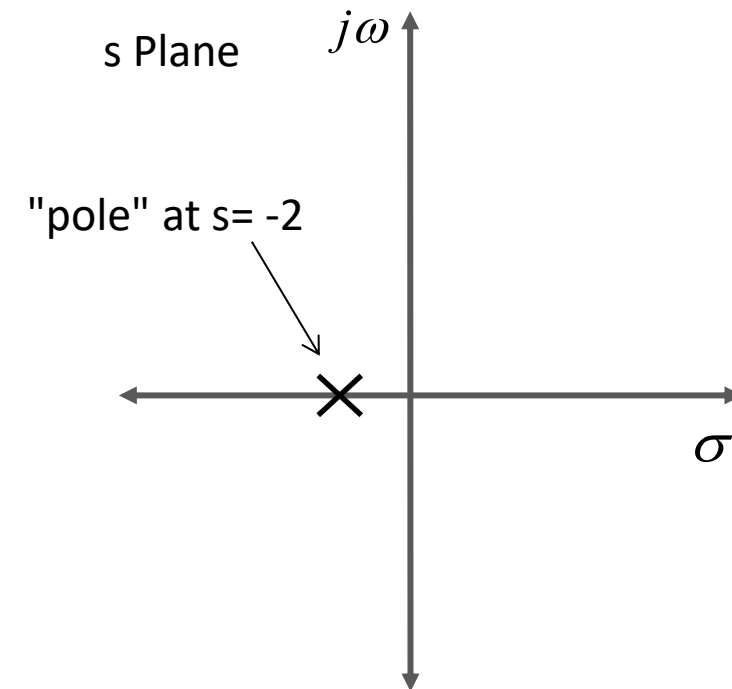
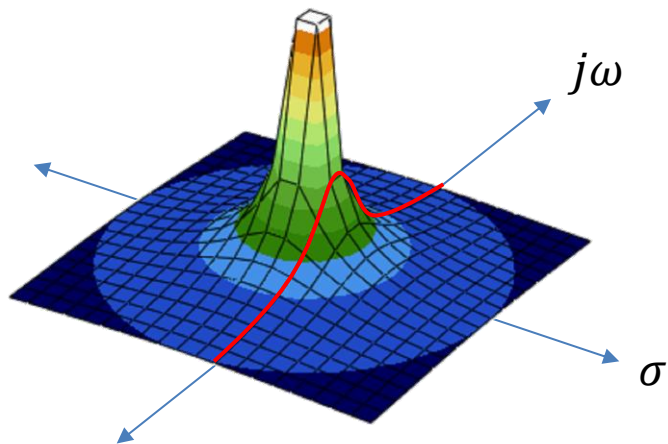
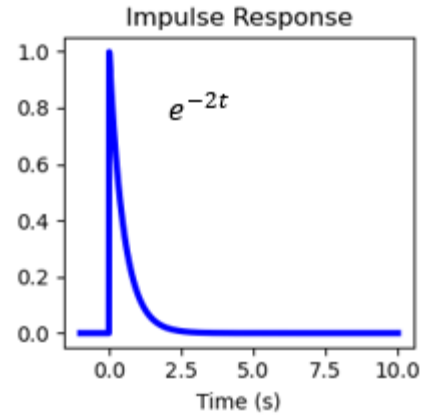
Graphing Magnitude of the Laplace Transform:



The s-plane

$$x(t) = e^{-2t}$$

$$X(s) = \frac{1}{s + 2}$$

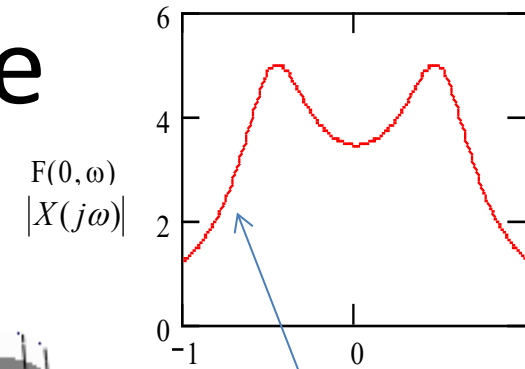


Filter Example

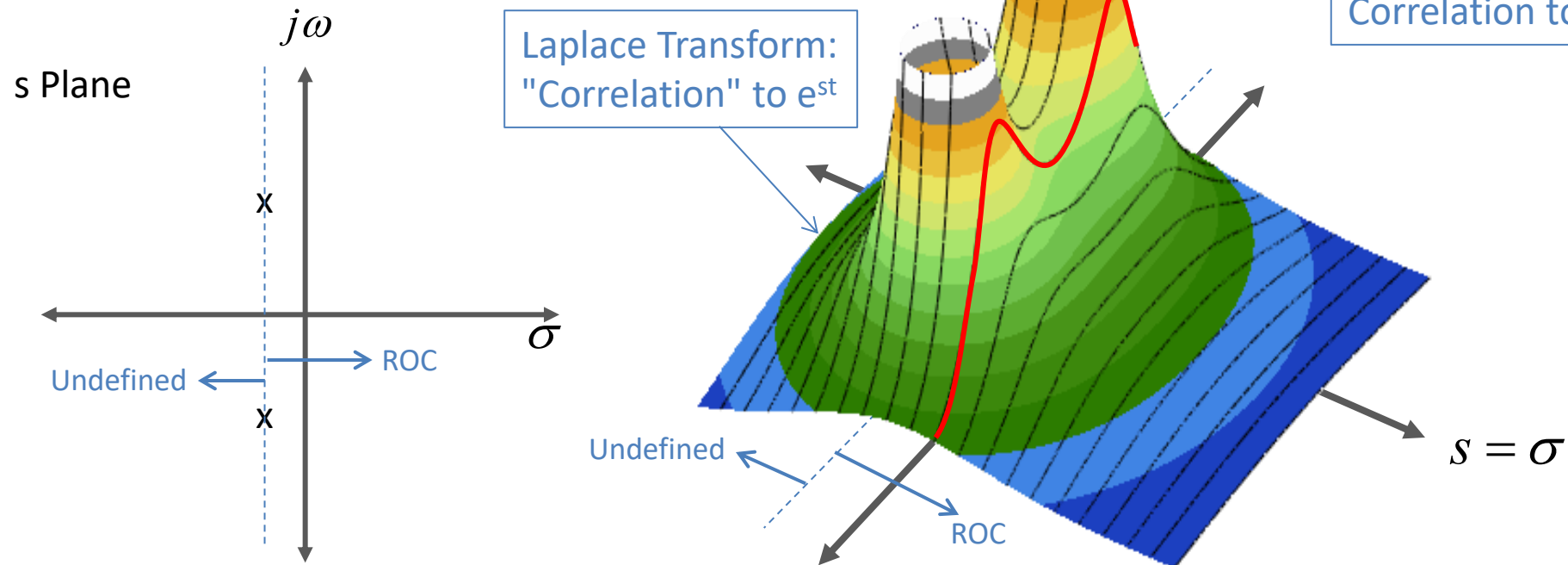


$$X(s) = \frac{1}{(s + .2 + j.5)(s + .2 - j.5)}$$

(poles at $s = -.2 \pm j.5$)



Fourier Transform:
Correlation to $e^{j\omega t}$

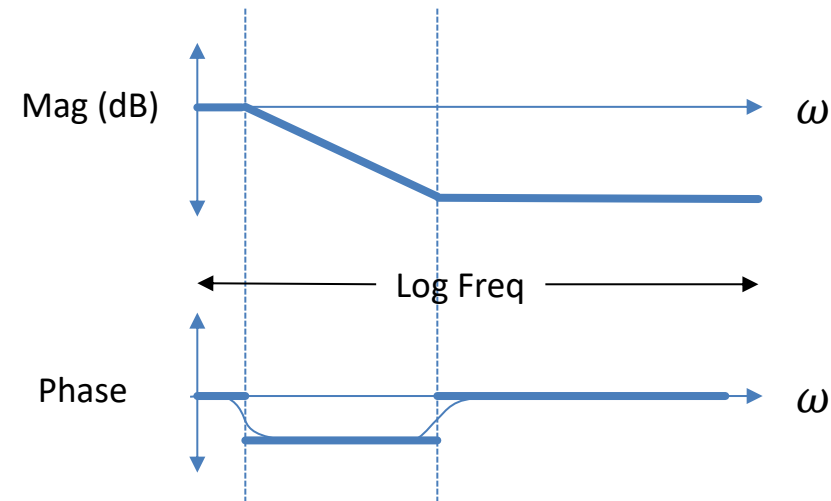
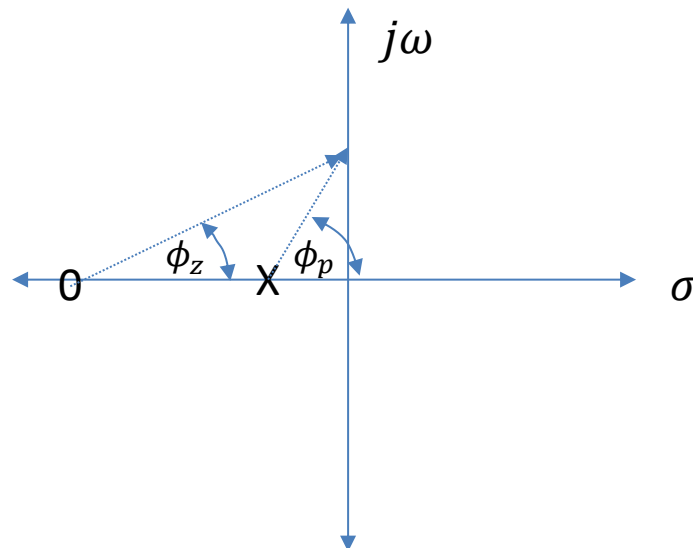


$$s = j\omega$$

Poles and Zeros

Each pole contributes -90° and -6 dB/octave to the freq response

Each zero contributes $+90^\circ$ and $+6$ dB/octave to the freq response



Fourier and Laplace – Discrete Time



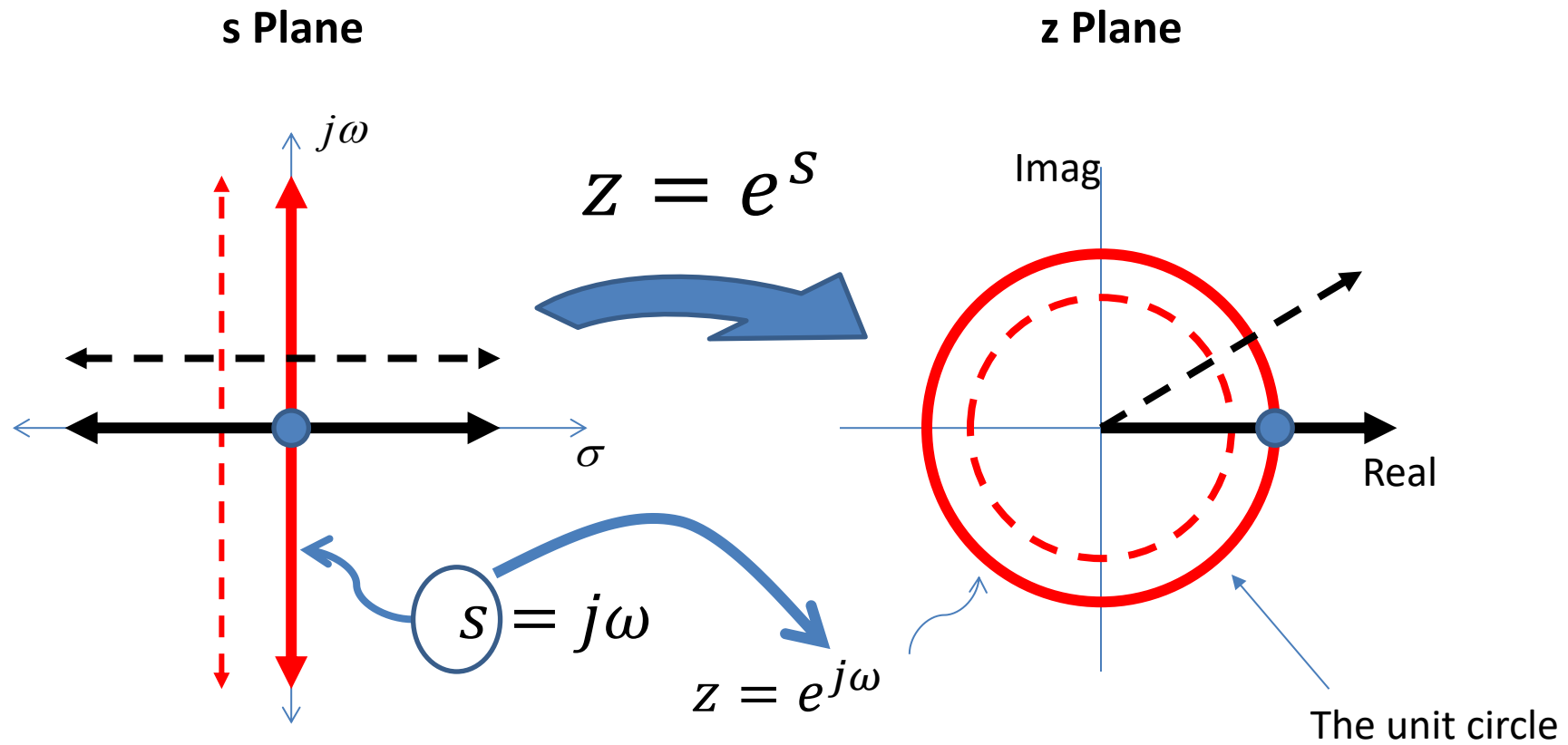
$$X(\omega) \equiv \int_{t=-\infty}^{\infty} x(t)e^{-j\omega t} dt \quad X(\omega) \equiv \sum_{n=-\infty}^{\infty} x[n]e^{-j\omega n}$$

DTFT

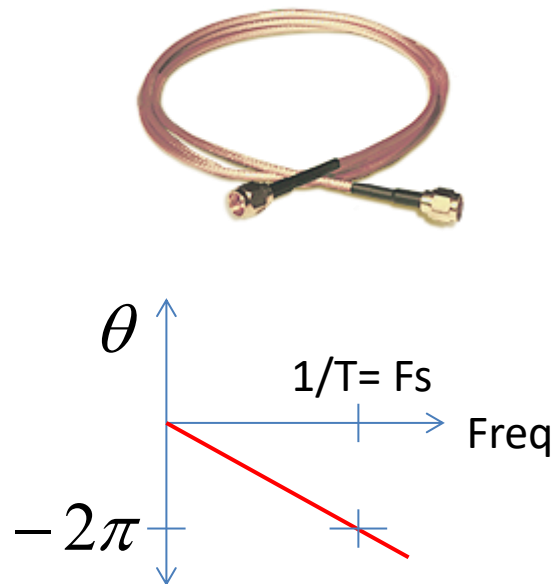


$$X(s) \equiv \int_{t=0}^{\infty} x(t)e^{-st} dt \quad X(s) \equiv \sum_{n=0}^{\infty} x[n]e^{-sn} \quad \rightarrow z = e^s \rightarrow X(z) \equiv \sum_{n=0}^{\infty} x[n]z^{-n}$$

z Transform!



$$z^{-1}$$

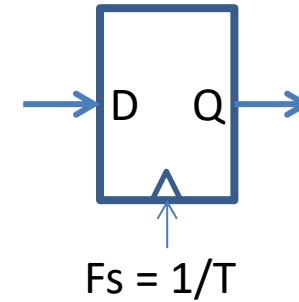


Laplace Transform for a constant delay of time T , $x(t-T)$

$$e^{sT} = z$$

$$e^{-sT} = z^{-1}$$

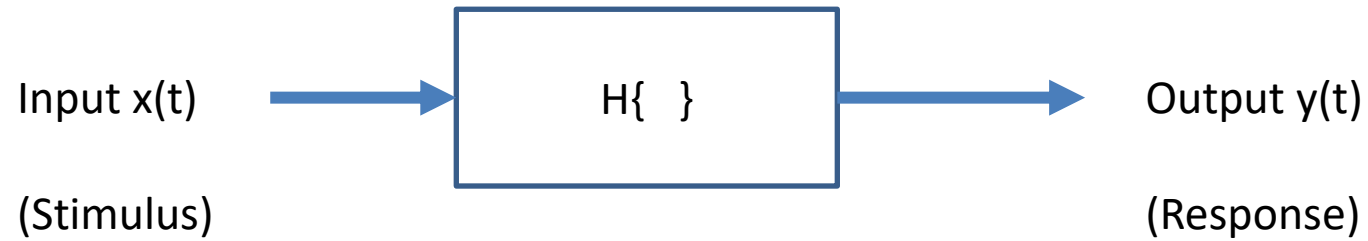
z Transform for a unit sample delay, where $F_s = 1/T$





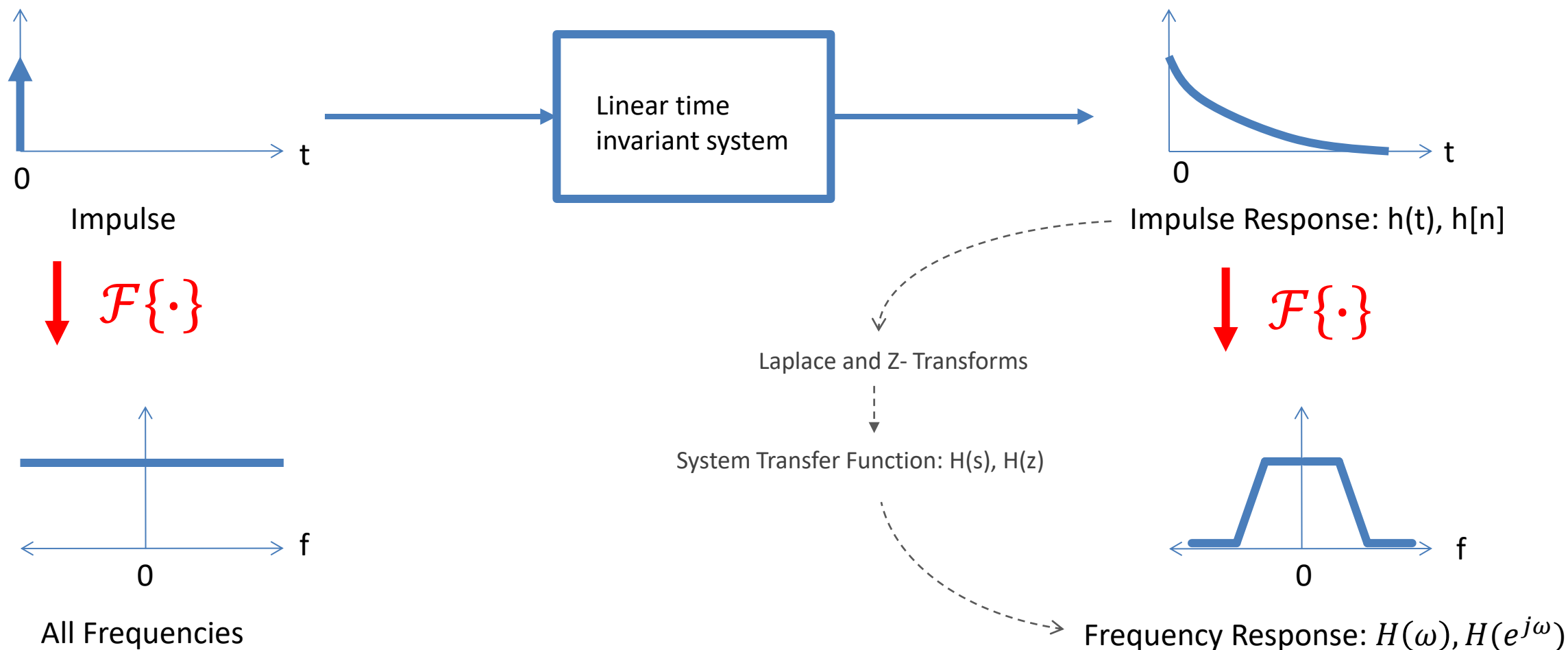
CONTROL SYSTEMS

Systems

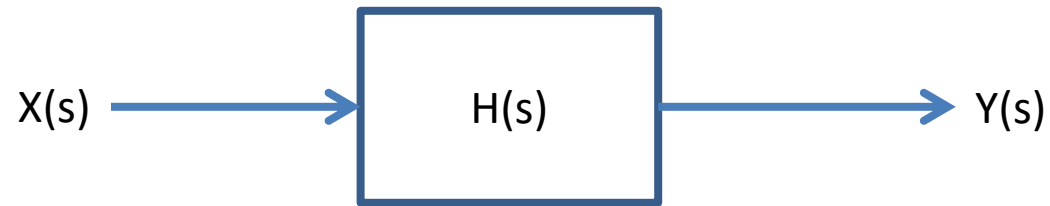


We will be covering the (simpler) world of **linear, causal, time-invariant** systems

Impulse Response and Frequency Response

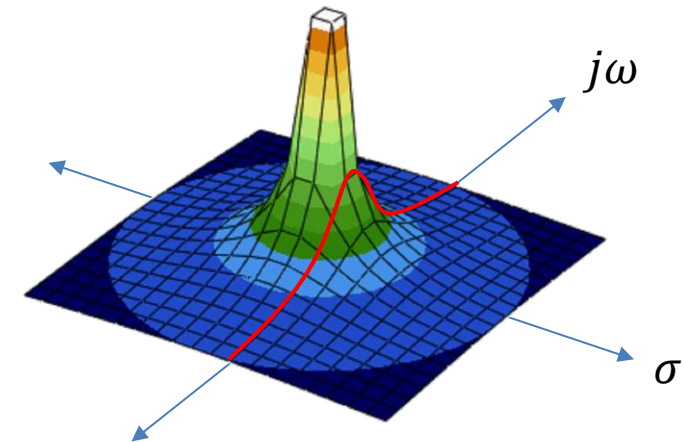
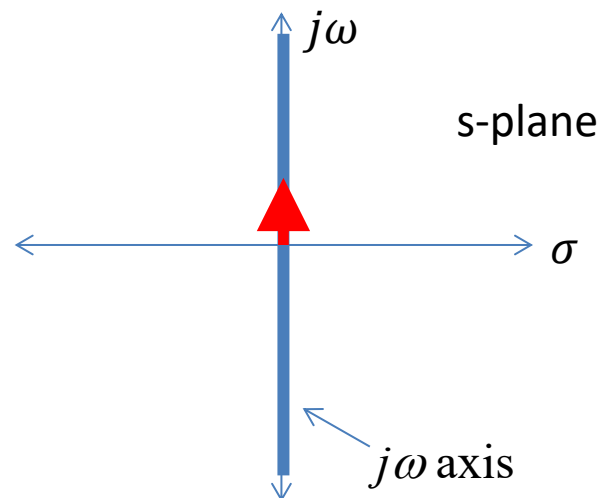


Transfer Functions



Stability: pole locations
Dynamics, impulse response, step response

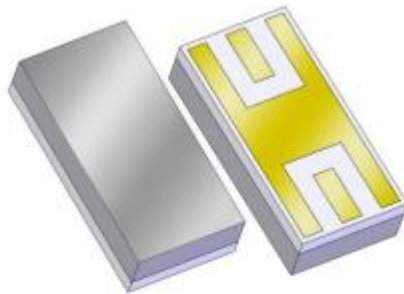
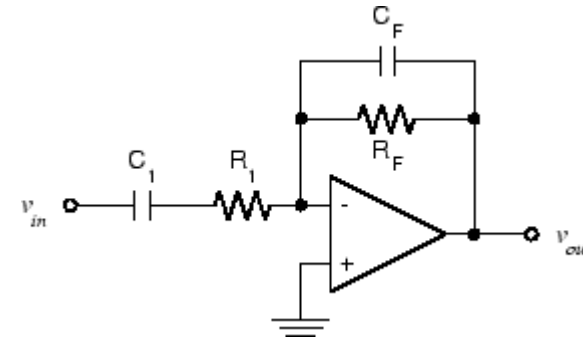
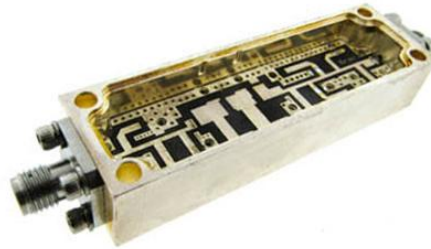
When $H(s)$ is evaluated when $s = j\omega$: Frequency Response



Transfer Functions

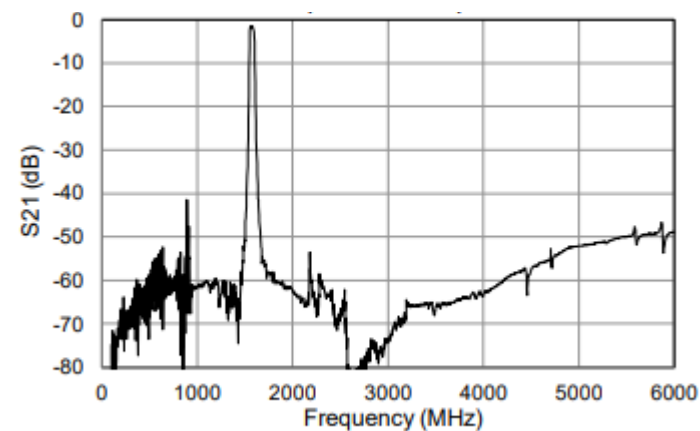


Example Bandpass Analog Filters with a Transfer Function given as $H(s)$

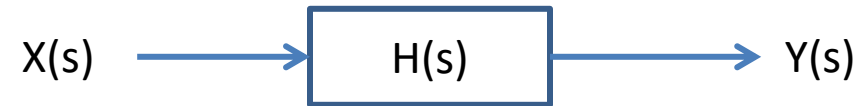


CSP: 3.26 X 1.60 X 0.84 mm
(Qorvo 880273 GPS Bandpass Filter)

Frequency Response: $H(j\omega)$



Loop Transfer Function

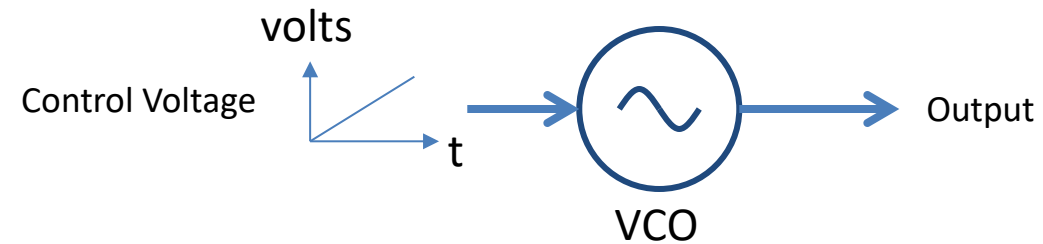
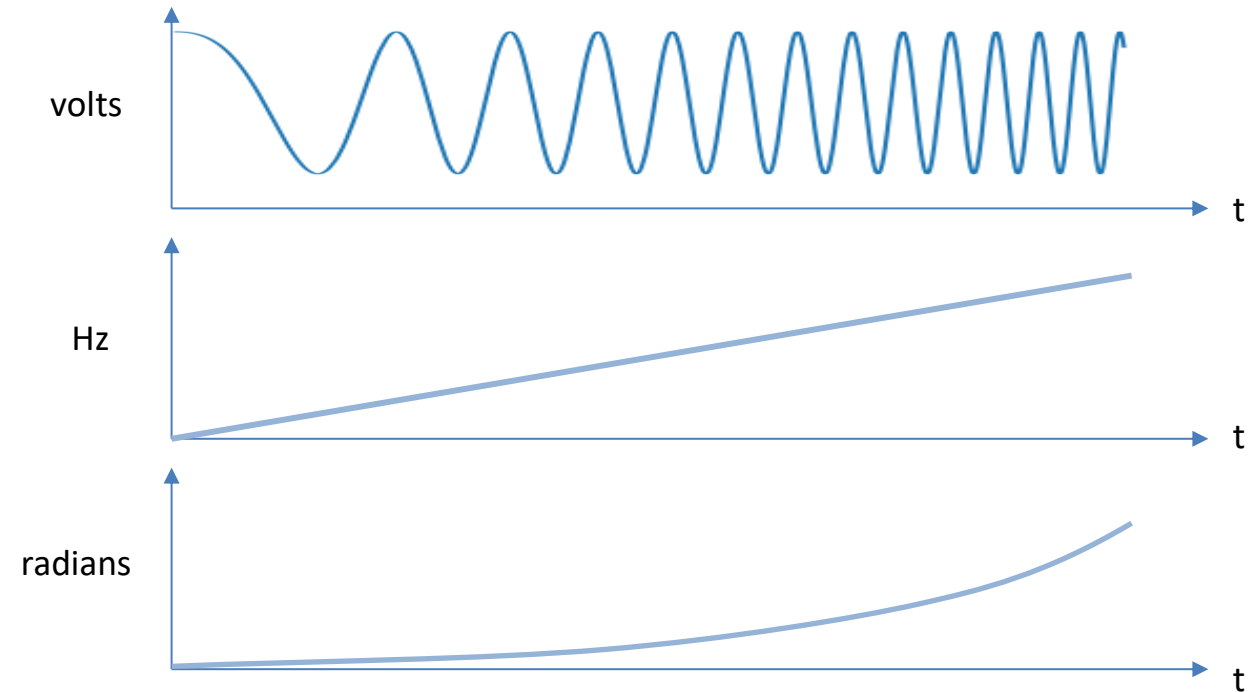


Understand the transfer function of each element involved

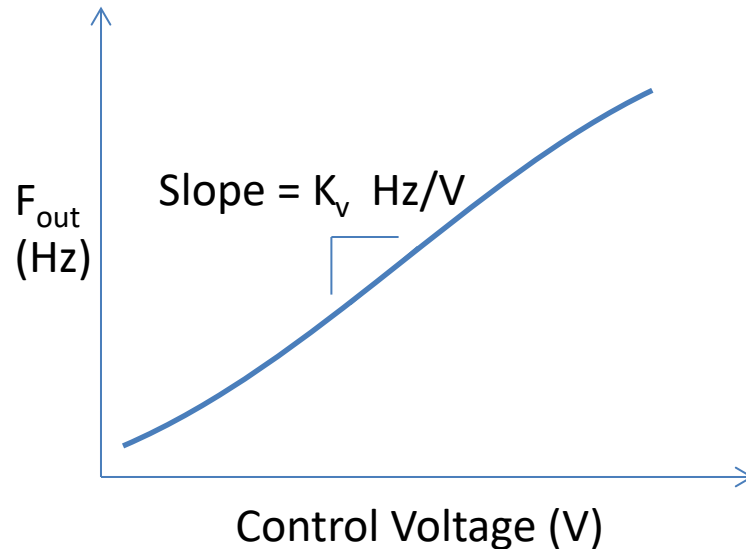
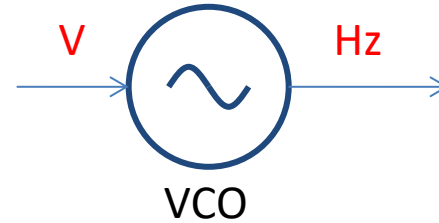
Pay careful attention to units to derive correct transfer function

Example: VCO ...

VCO Units

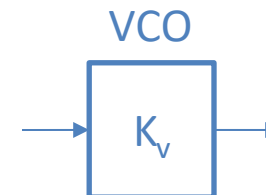


VCO Transfer Function



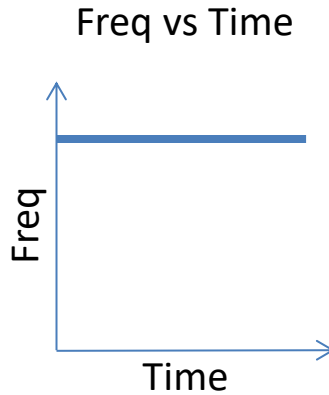
When frequency is unit of interest,
VCO converts Volts to Hz

Transfer function = K_v (Hz/V)

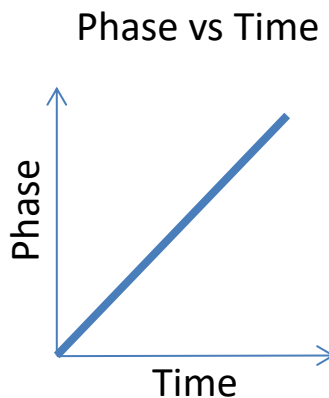


Constant "Gain" of K_v , and unit
conversion from Volts to Hz

VCO Transfer Function



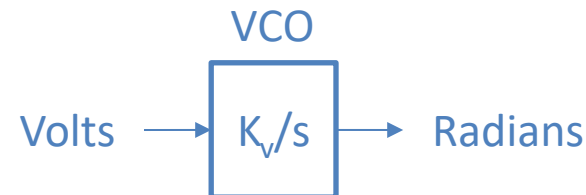
↓ $\theta(t) = \int_0^t f(t) dt$



$$\theta(t) = \int_0^t f(t) dt \xrightarrow{\mathcal{L}} \frac{1}{s} F(s)$$

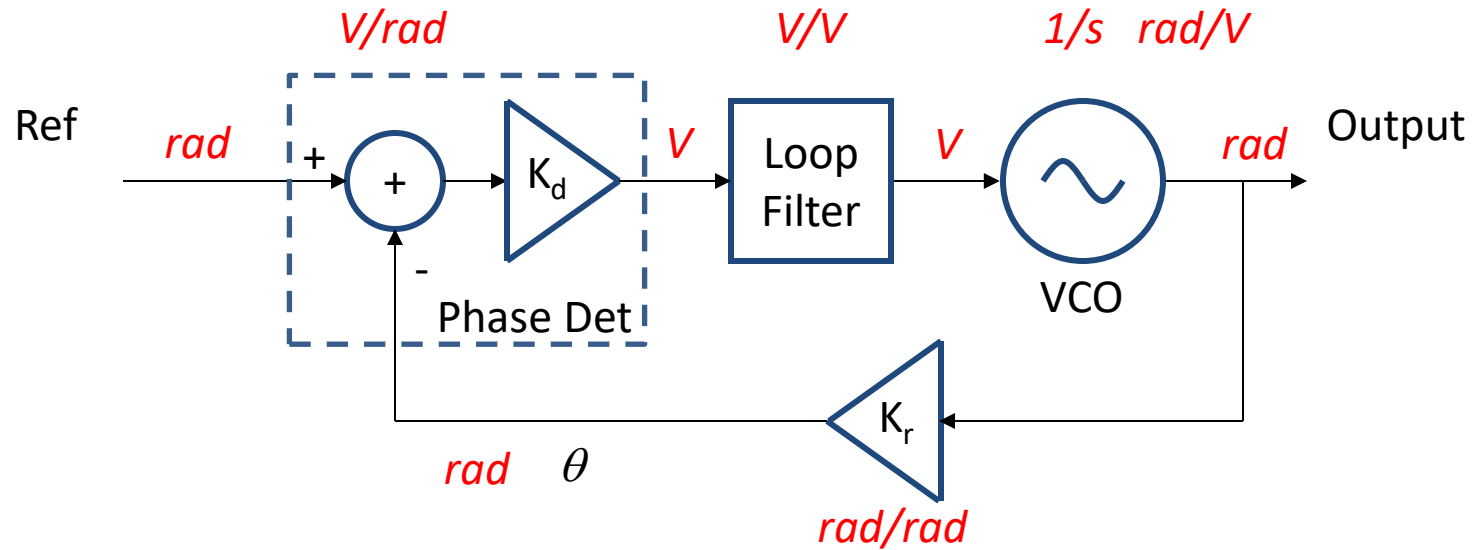
When phase is unit of interest, VCO is a "lowpass" converting Volts to Radians

Transfer function = K_v/s (Radians/V)



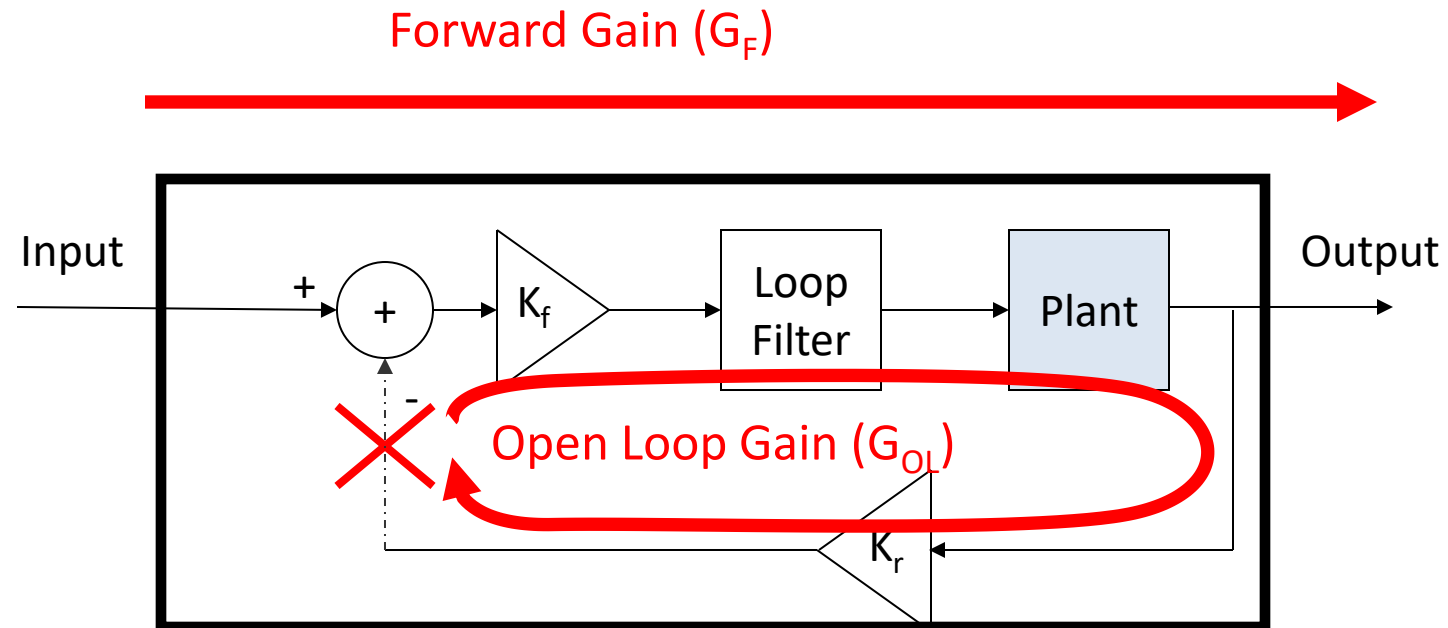
Lowpass filter with gain K_v/s and unit conversion from Volts to Radians

Phase Lock Loop



Keep track of units!

Loop Transfer Function



Closed Loop Gain:

$$G_{CL} = \frac{G_F}{1 + G_{OL}}$$

Loop Transfer Function



Stability:

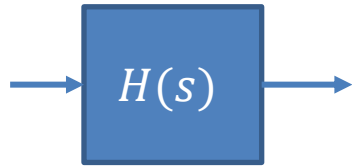
$G_{CL}(s)$ is unstable for any poles in the RHP

$$G_{CL} = \frac{G_F}{1 + G_{OL}}$$

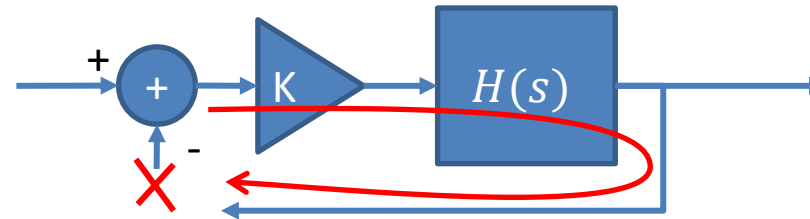
Characteristic Equation

The zeros (roots) of $1+G_{OL}(s)$ are the poles of $G_{CL}(s)$

Loop Transfer Function

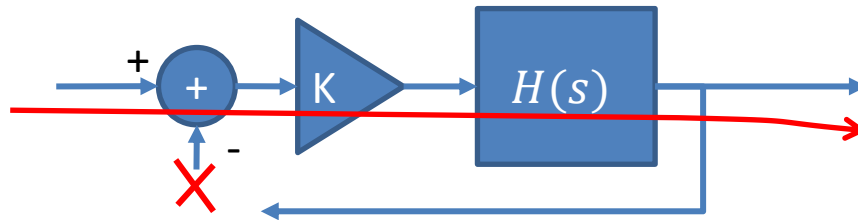


Open Loop Transfer Function



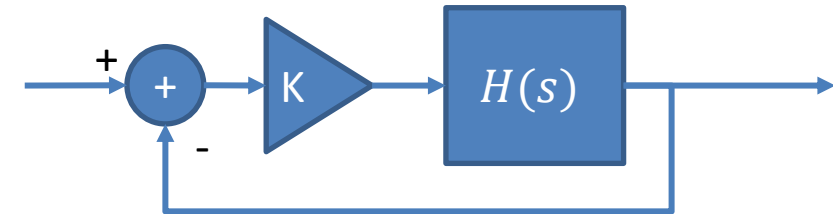
$$G_{OL}(s) = KH(s)$$

Forward Path Transfer Function



$$G_F(s) = KH(s)$$

Closed Loop Transfer Function

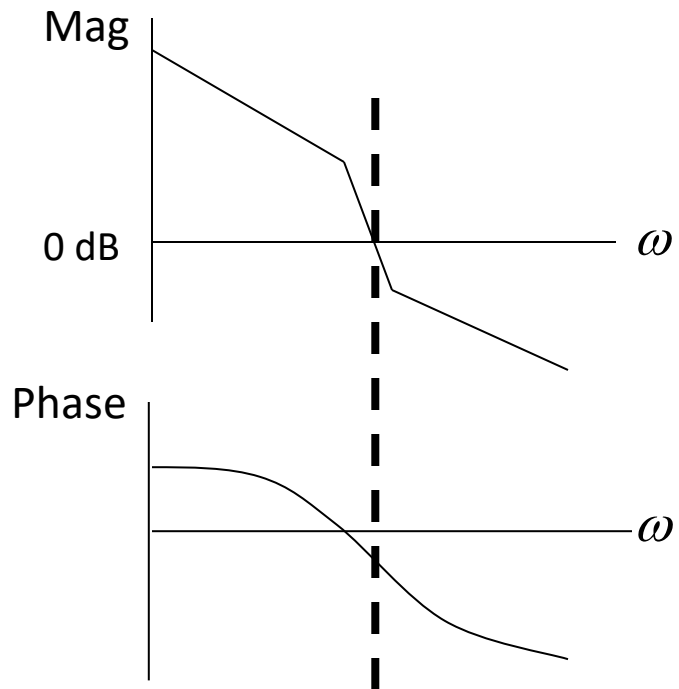


$$G_{CL}(s) = \frac{G_F(s)}{1 + G_{OL}(s)} = \frac{KH(s)}{1 + KH(s)}$$

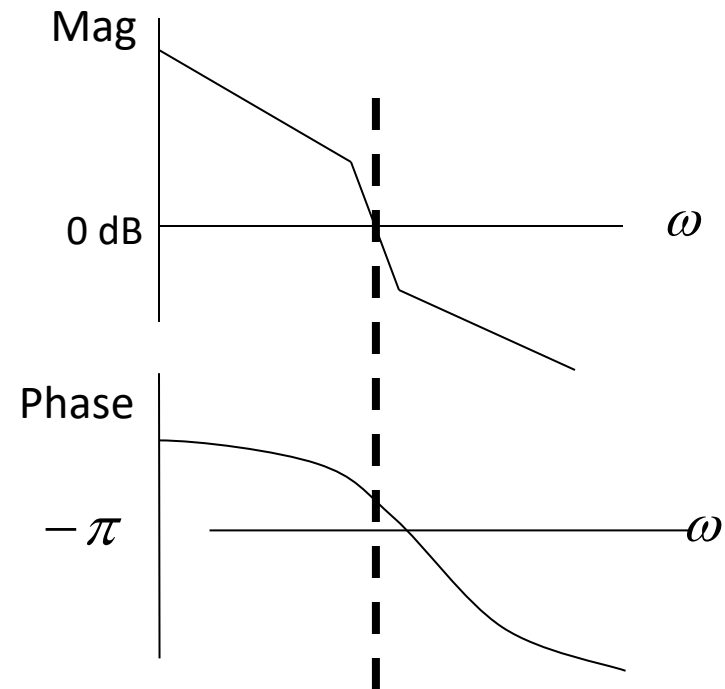
Stability – Bode Plot



Determined from **Open Loop Gain**: When $|G_{OL}| \geq 1, \angle G_{OL} < -\pi$

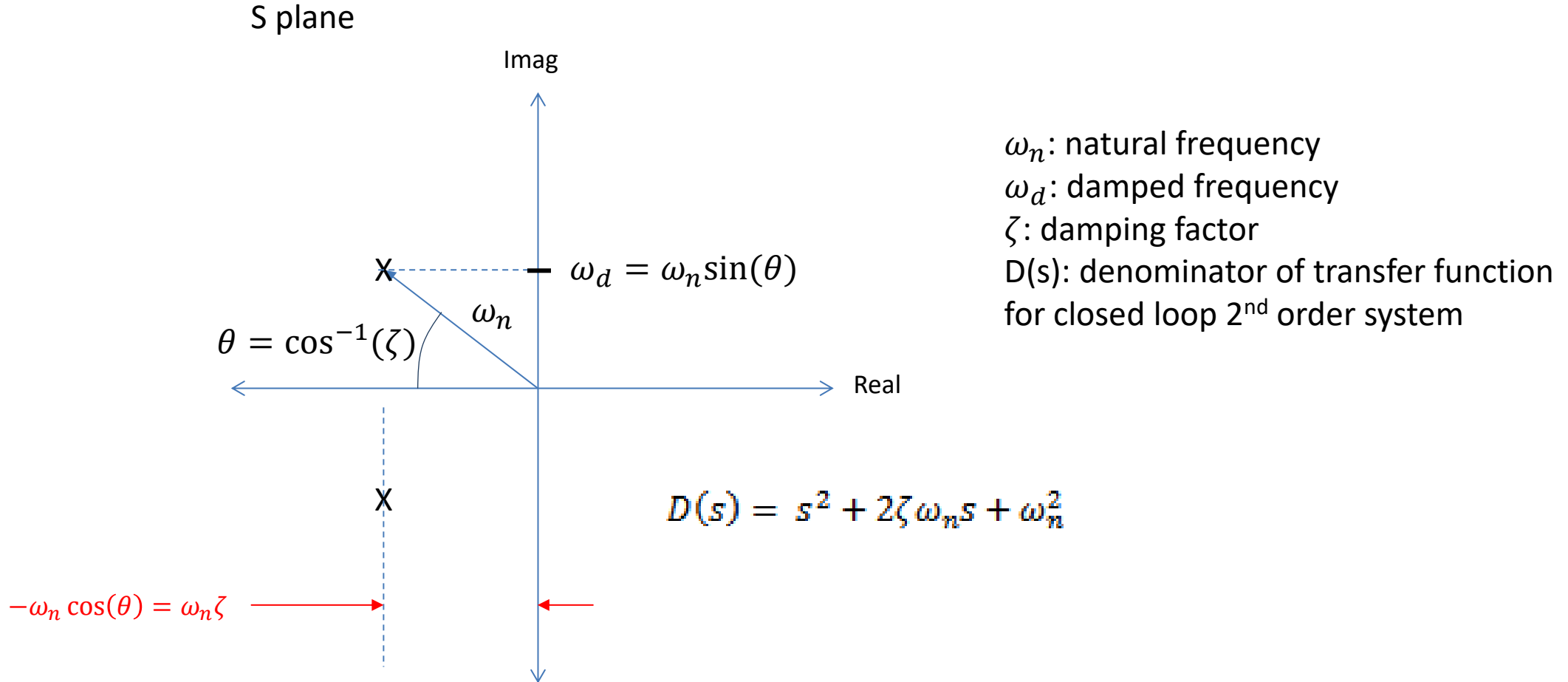


Unstable!

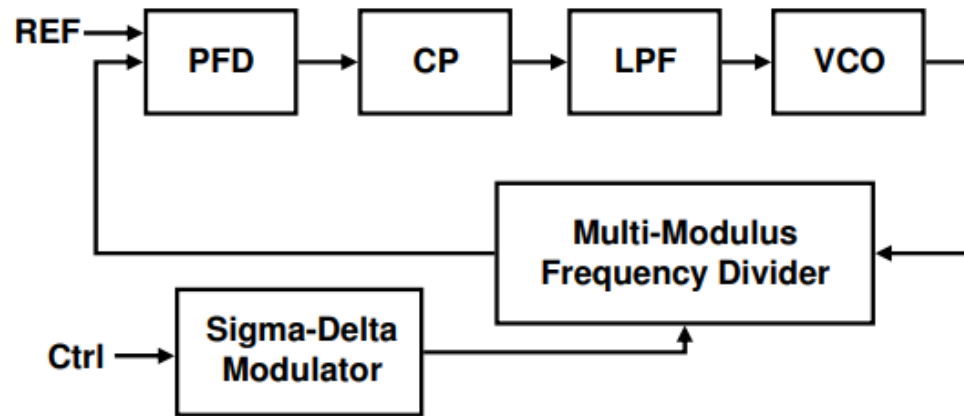


Stable!

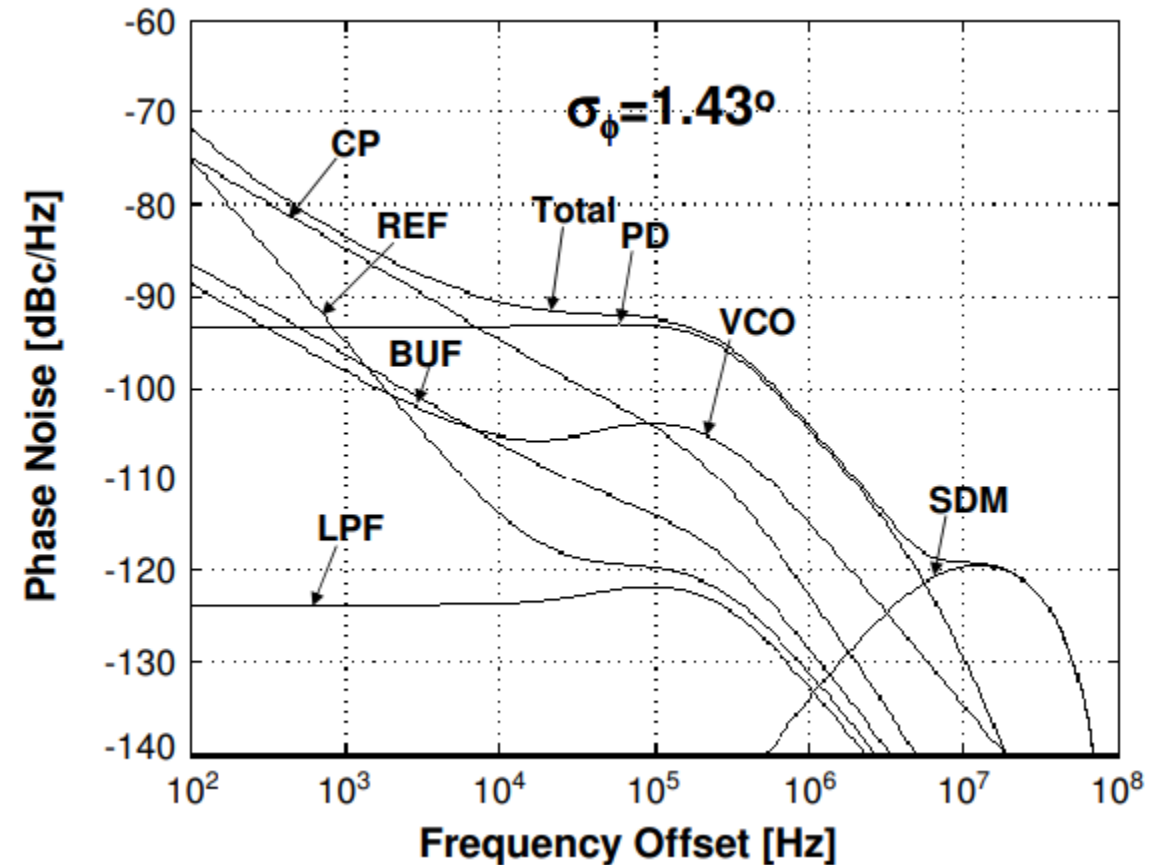
Closed-Loop Poles for 2nd Order System



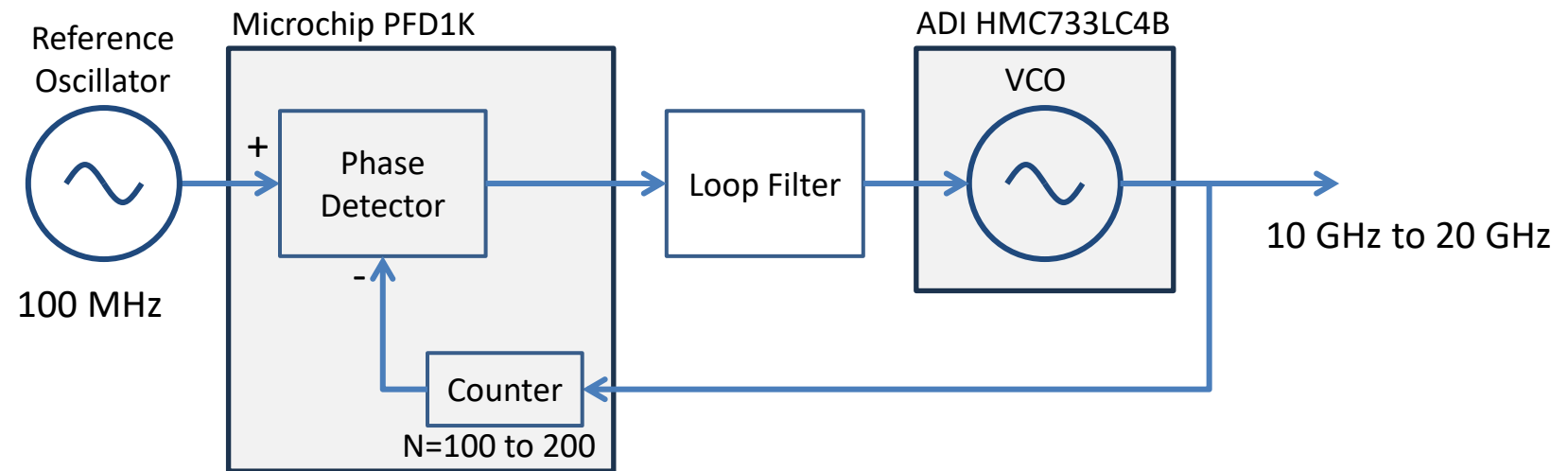
Composite Phase Noise in Frac-N PLL



Source: "Analytical Phase Noise Modeling and Charge Pump Optimization for Fractional-N PLLs", Frank Herzel et al, IEEE Transactions on Circuits and Systems-I: Regular Papers, Vol. 57, No. 8, August 2010.

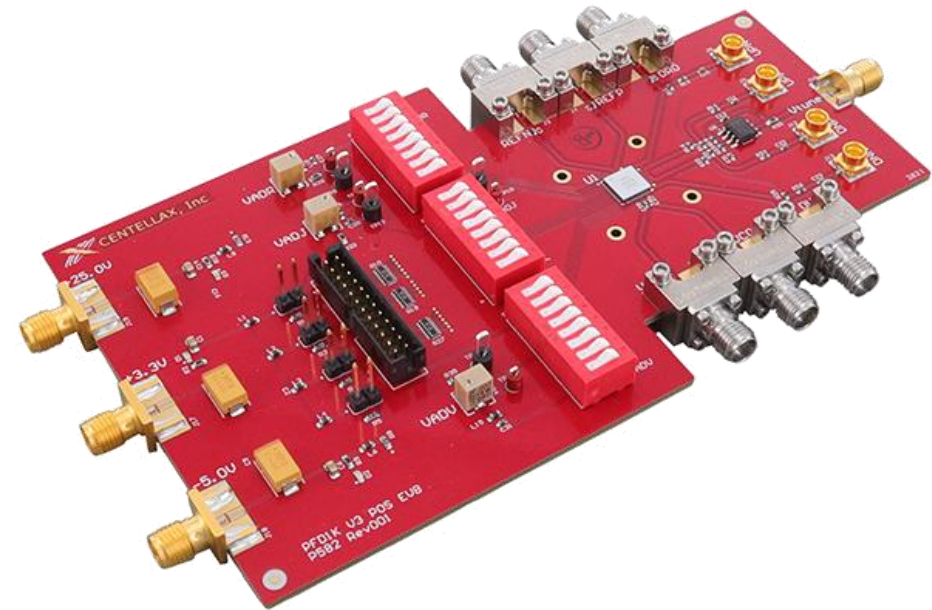
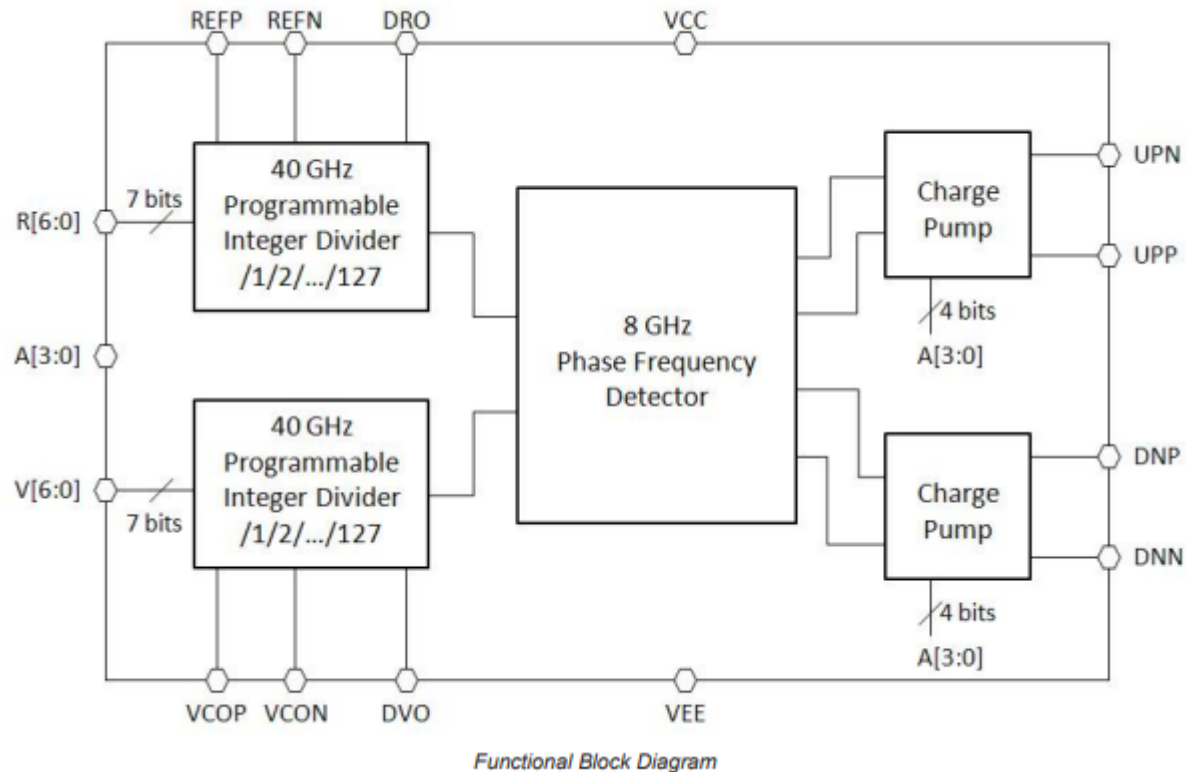


EXAMPLE ANALOG PLL



Integer-N Phase Lock Loop

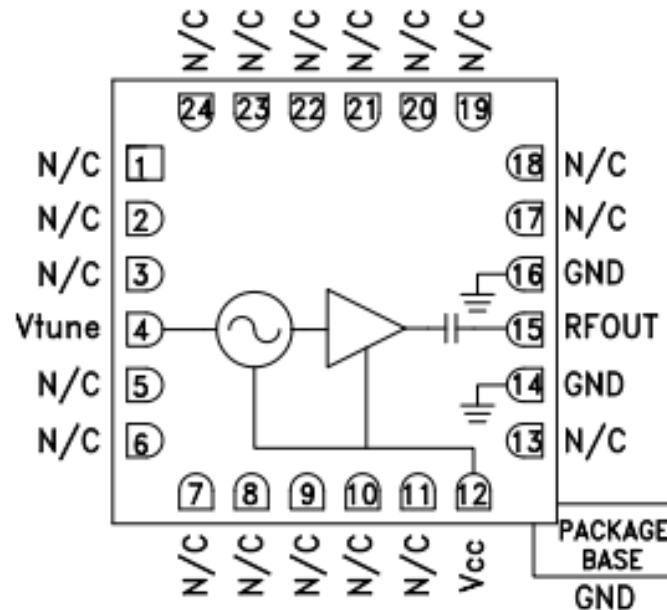
Microchip 8 GHz Phase Freq Detector PFD1K



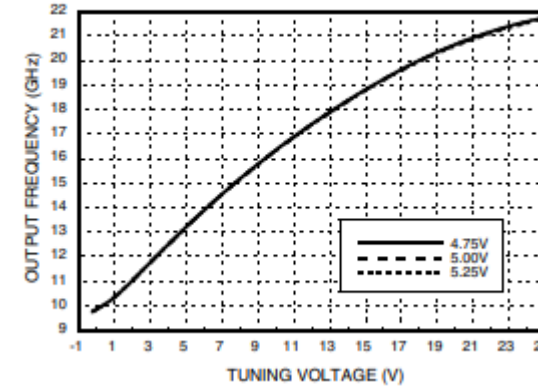
ADI 10 – 20 GHz VCO HMC733LC4B



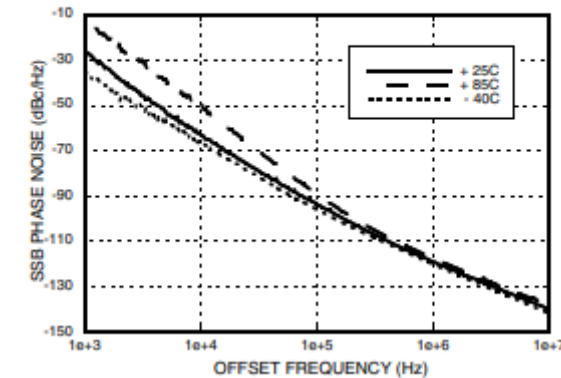
Functional Diagram



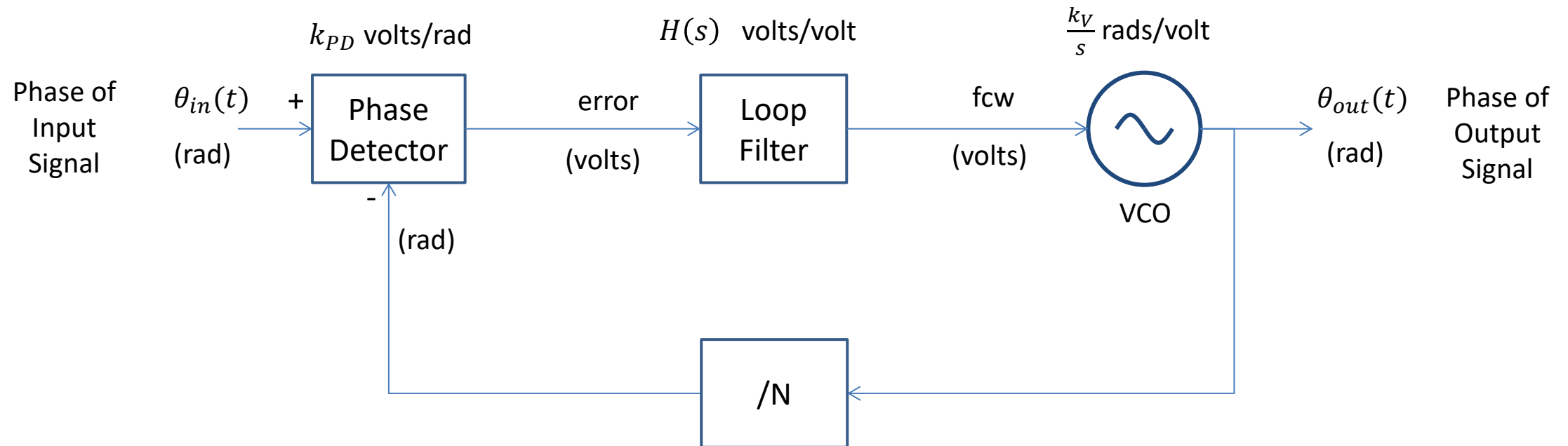
Frequency vs. Tuning Voltage, $T = +25^\circ\text{C}$



**Typical SSB Phase Noise vs. Temperature
 $V_{tune} = +10V$**



Analog PLL Loop Model



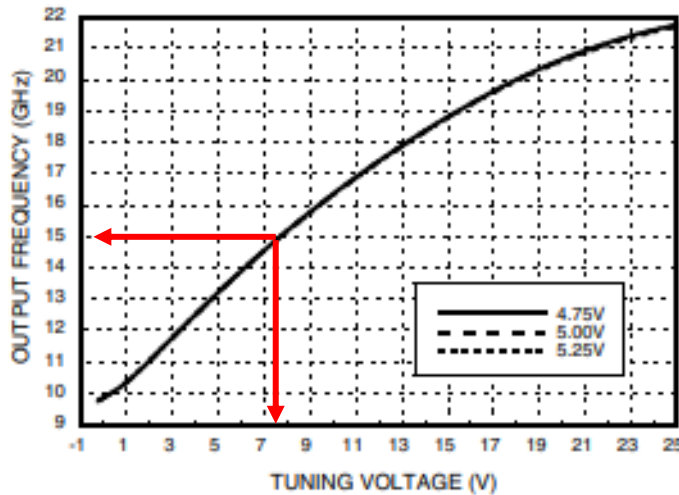
VCO Loop Gain

slope of Freq vs Tuning is $\longrightarrow k_V$

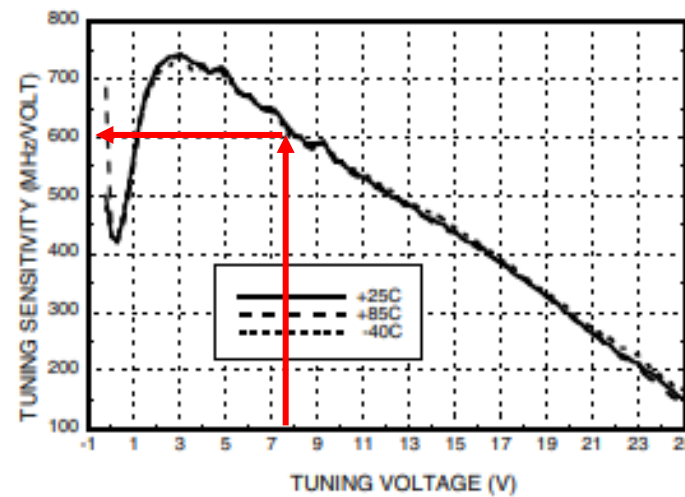
VCO Loop Model is

$$\frac{k_V}{s} \quad \text{rads/volt}$$

Frequency vs. Tuning Voltage, $T = +25^\circ\text{C}$



Sensitivity vs. Tuning Voltage, $V_{cc} = +5V$, $T = +25^\circ\text{C}$



At 15 GHz, slope is 600 MHz/Volt. At this operating point $k_V = 2\pi 600e6 \text{ (rad/sec)/volt}$

Phase Detector Gain

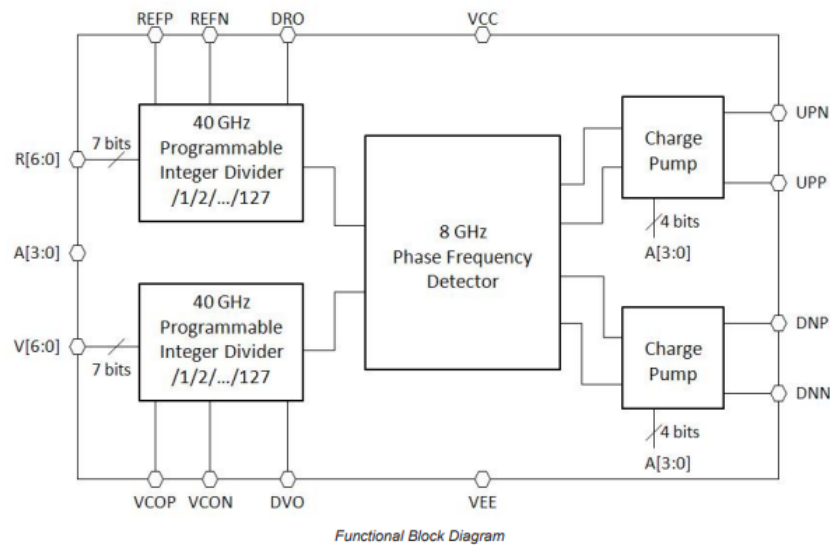
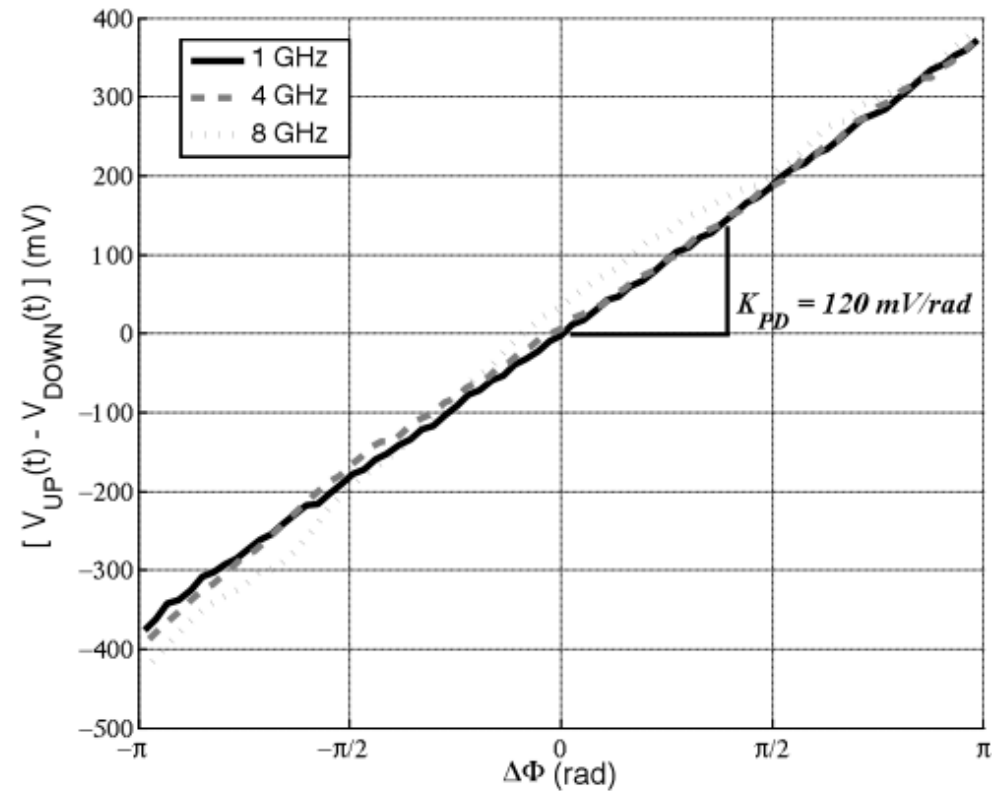
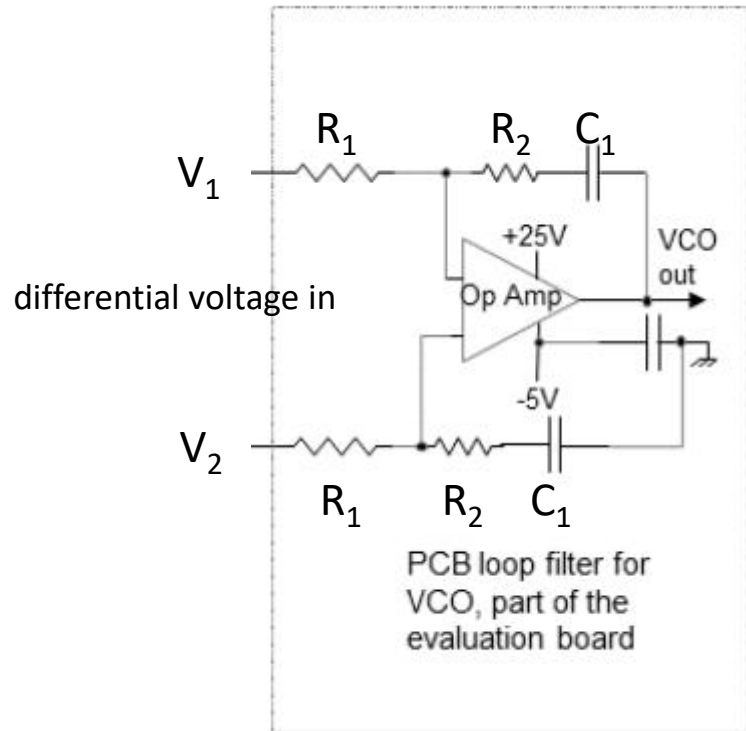


Figure 1-7. Diff. Output Voltage vs. Frequency (0 dBm Pin)



Differential PI Loop Filter



Differential amplifier w/o C_1 is $V_{out} = \frac{R_2}{R_1} (V_2 - V_1)$

$$V_{out} = \frac{R_2 + \frac{1}{sC_1}}{R_1} (V_2 - V_1)$$

$$\frac{V_{out}}{V_{in}} = H(s) = \frac{1 + sR_2C_1}{sR_1C_1} = \frac{1 + s\tau_2}{s\tau_1}$$

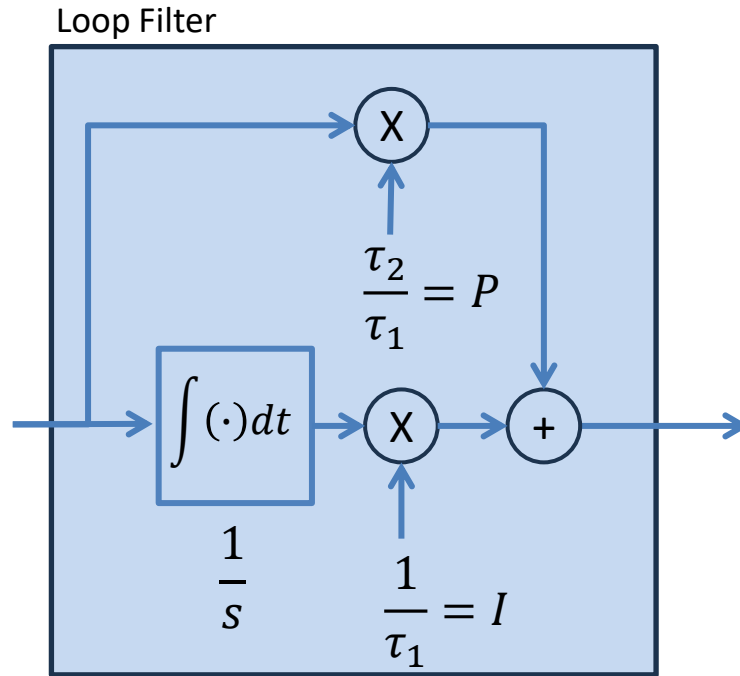
$$H(s) = \frac{1}{s} \frac{1}{\tau_1} + \frac{\tau_2}{\tau_1} = \frac{1}{s} I + P$$

PI Loop Filter

$$H(s) = \frac{1 + s\tau_2}{s\tau_1}$$

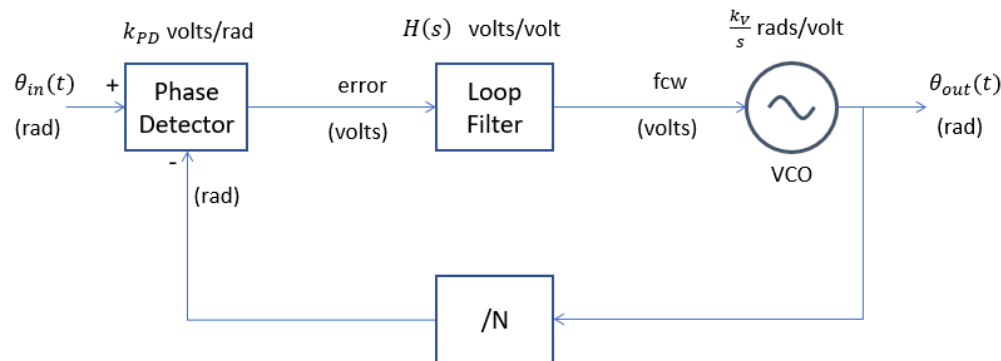
$$H(s) = \frac{1}{s} \frac{1}{\tau_1} + \frac{\tau_2}{\tau_1}$$

$$= \frac{1}{s} I + P$$



Open Loop Gain

$$G_{OL}(s) = \frac{k_V k_{PD}}{N s} H(s)$$

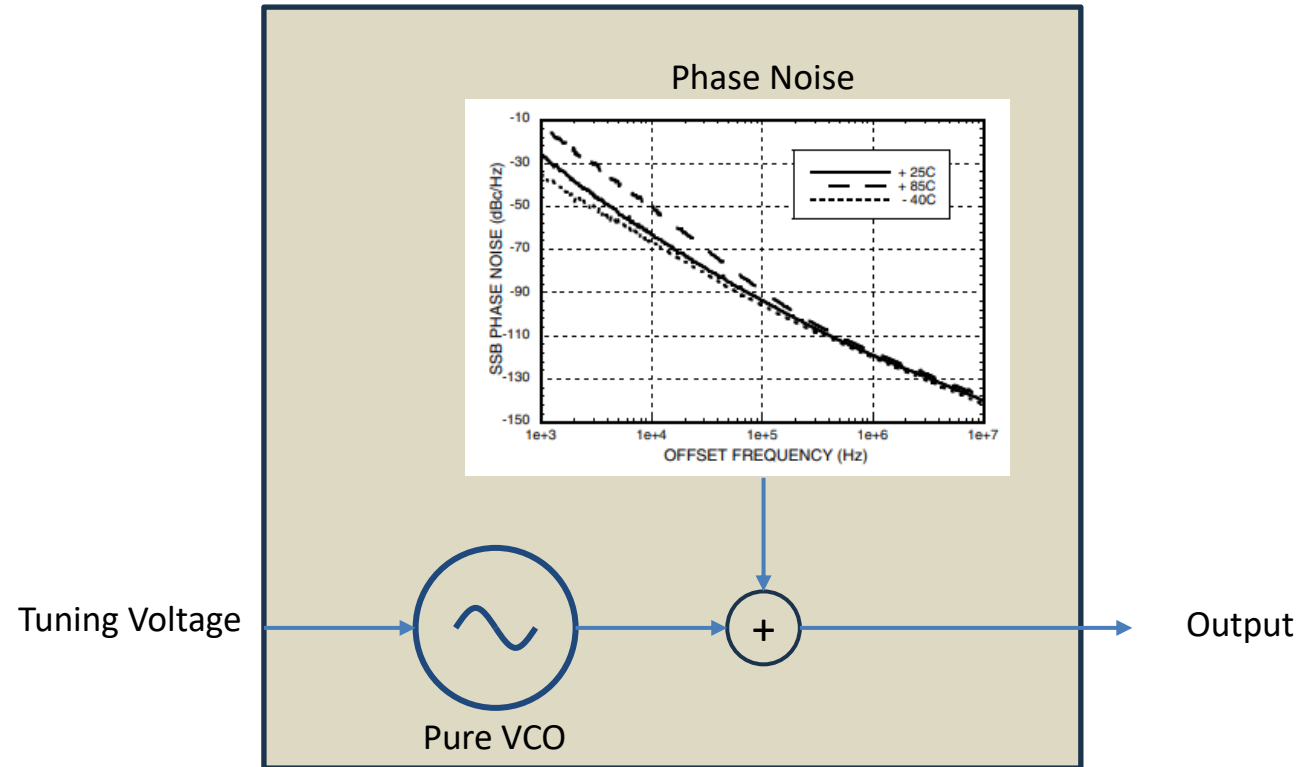


$$= \frac{k_V k_{PD}}{N s} \frac{1 + s\tau_2}{s\tau_1}$$

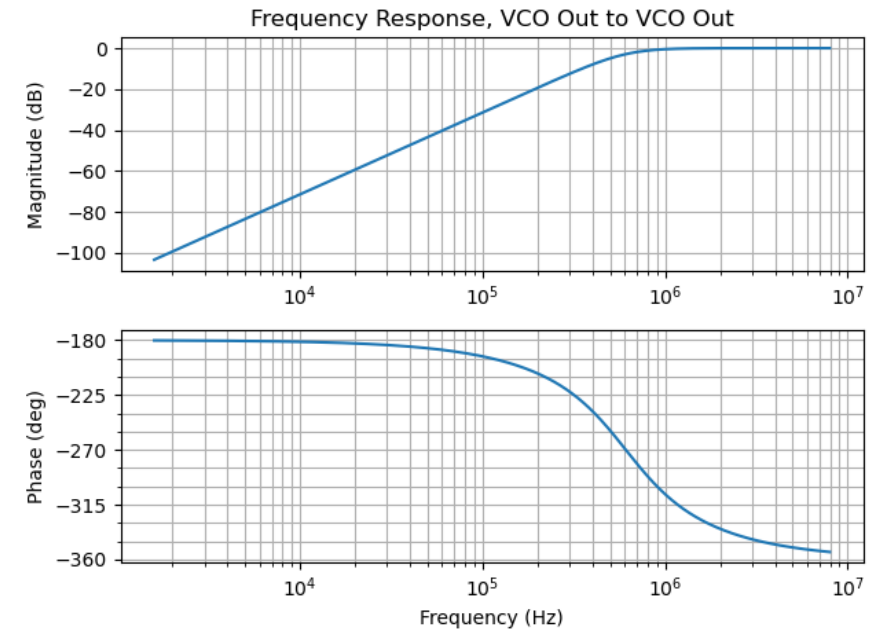
$$= \frac{k_V k_{PD}}{N \tau_1} \frac{1 + s\tau_2}{s^2}$$

See companion Marimo notebook `control_systems.py` for further simulation of this model

VCO Phase Noise Frequency Domain Model



Loop Transfer Function from Phase Noise Injection Node to Output



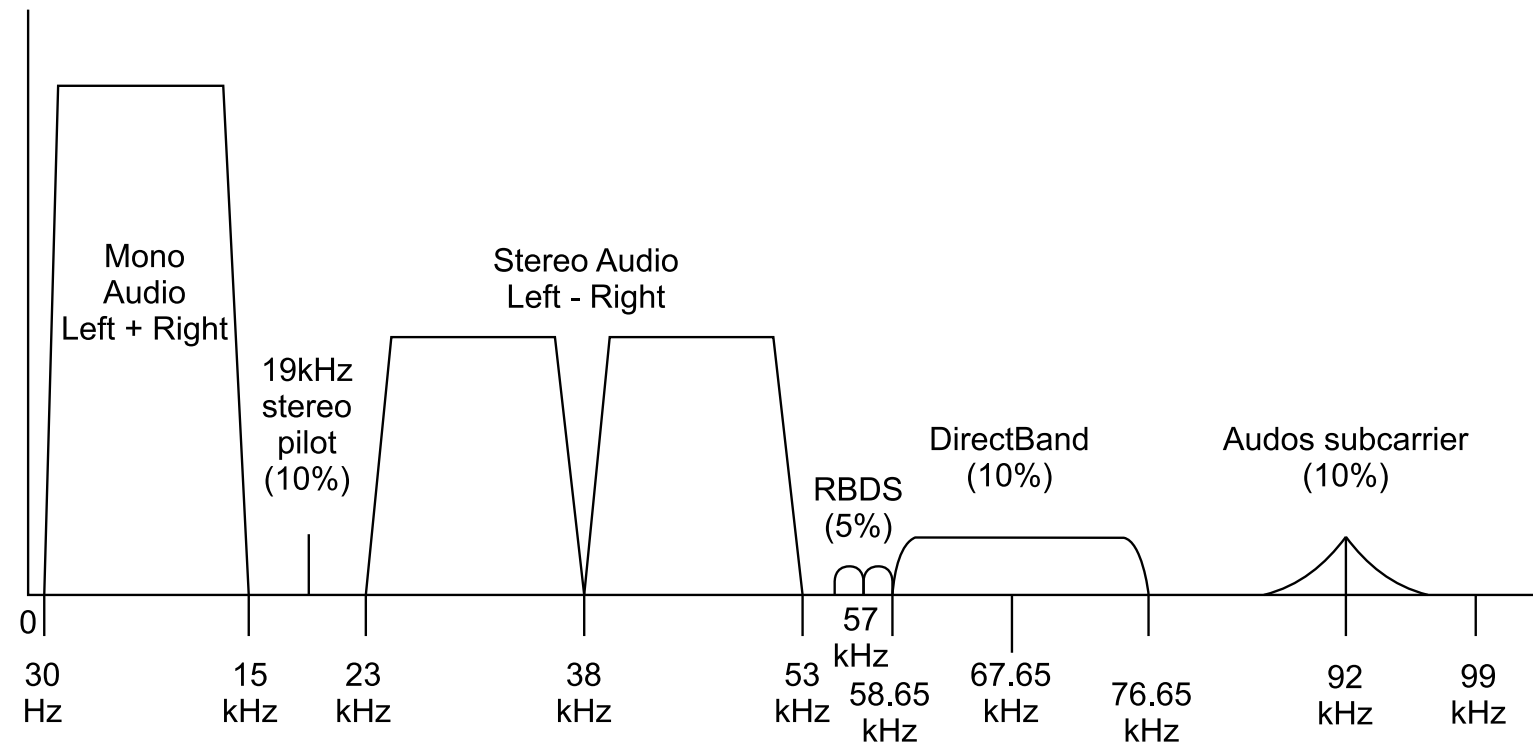
See companion Marimo notebook `control_systems.py` for further simulation of this model

EXAMPLE DIGITAL PLL

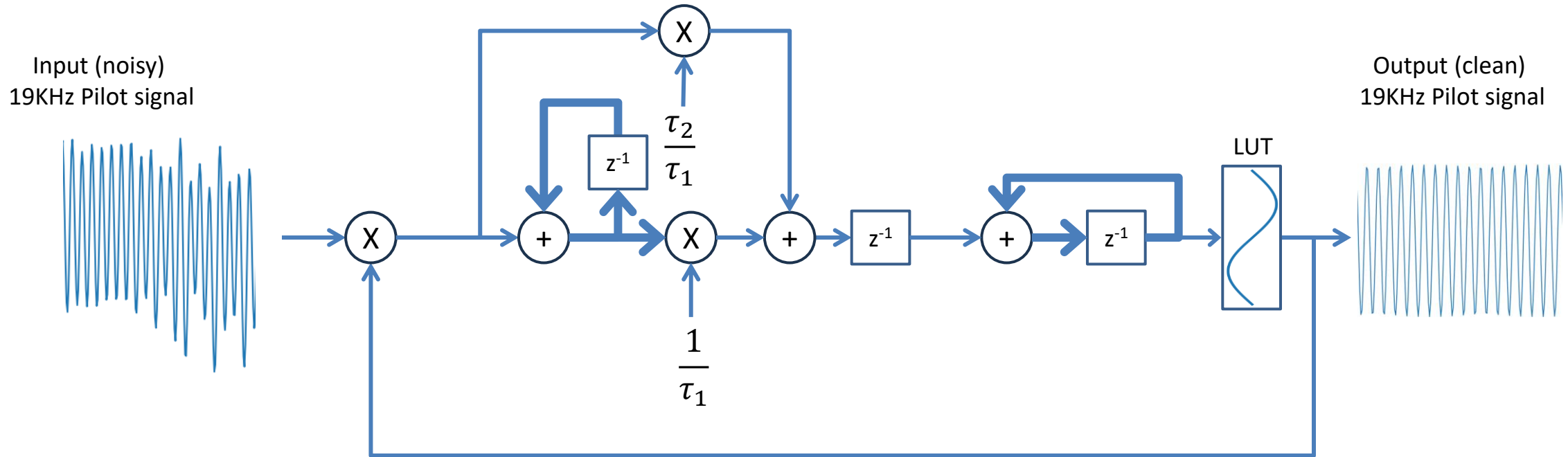
Digital PLL Example



Application: Tracking the 19 kHz pilot signal for FM broadcast

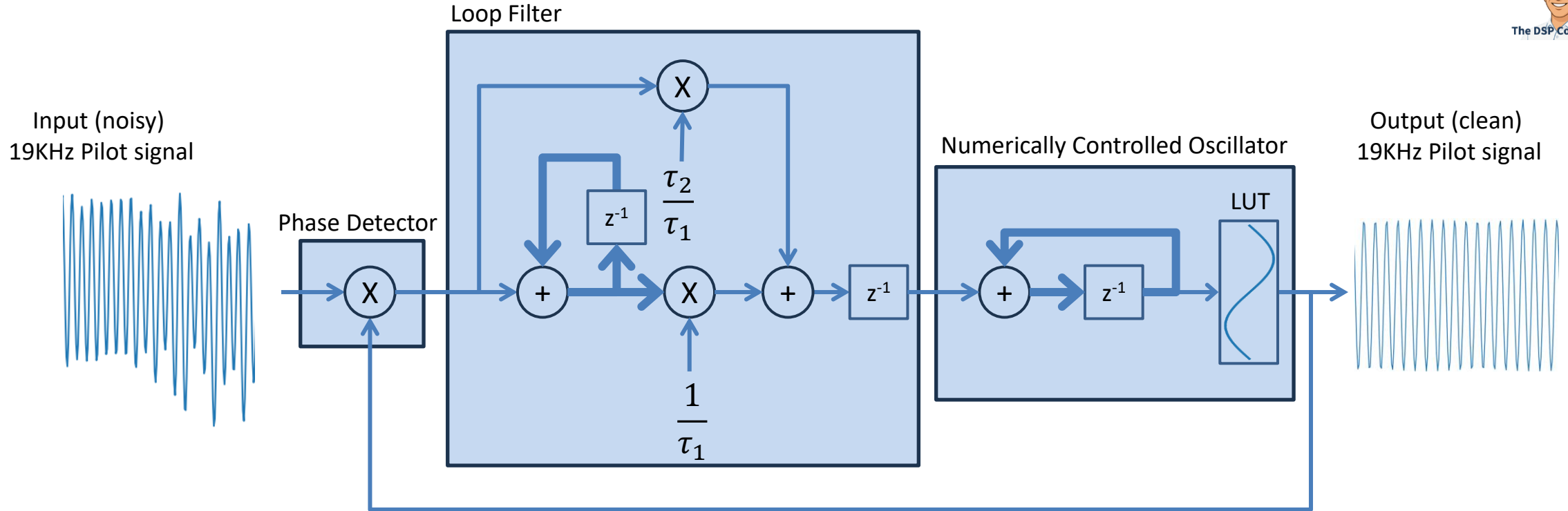


By Arthur Murray - http://en.wikipedia.org/wiki/File:RDS_vs_DirectBand_FM-spectrum2.png, Public Domain,
<https://commons.wikimedia.org/w/index.php?curid=5594246>



All Digital Phase Lock Loop

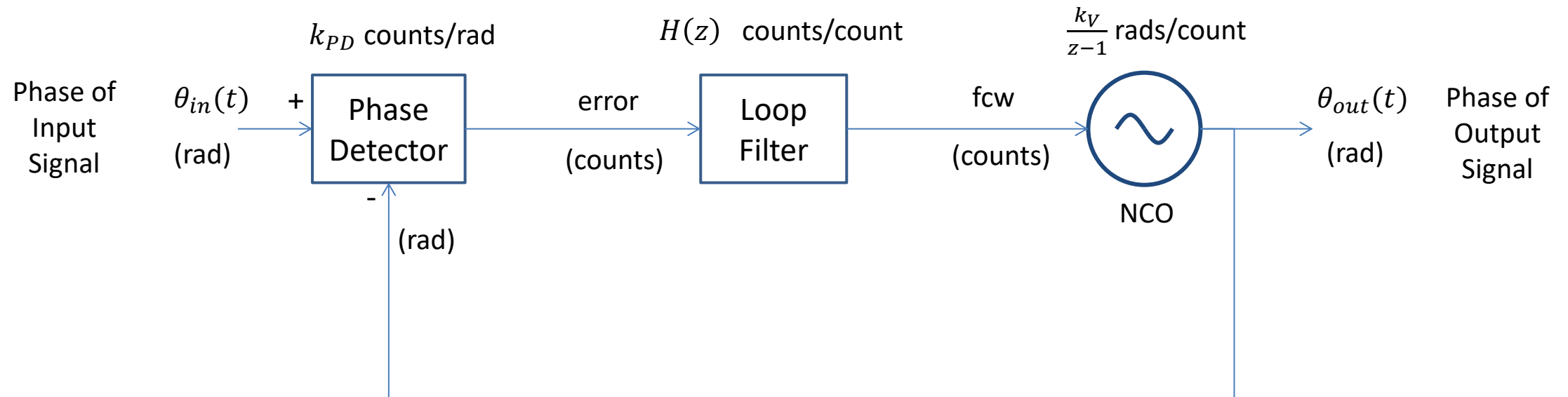
Update Rate 192 KHz



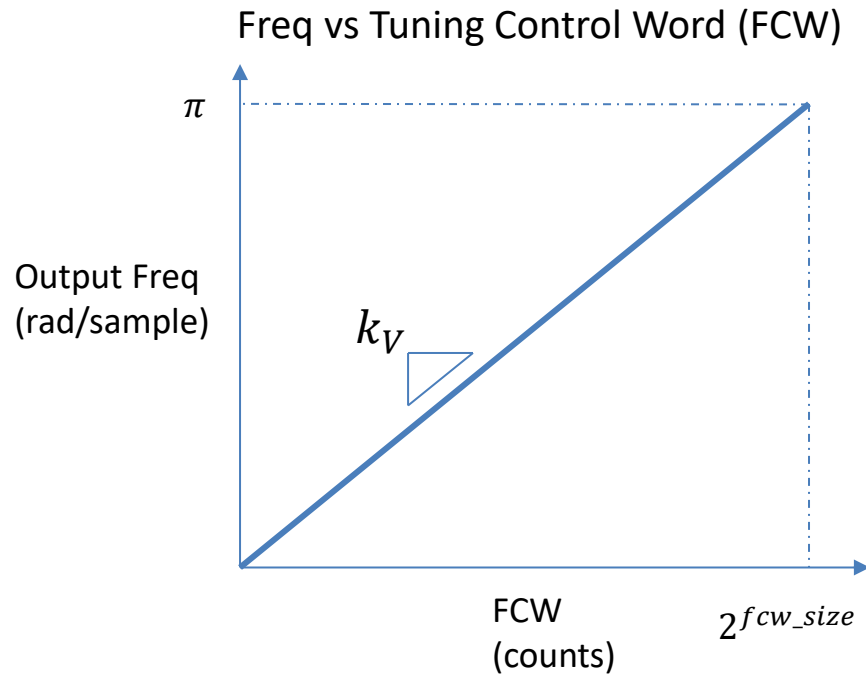
All Digital Phase Lock Loop

Update Rate 192 KHz

Digital PLL Loop Model



NCO Loop Gain

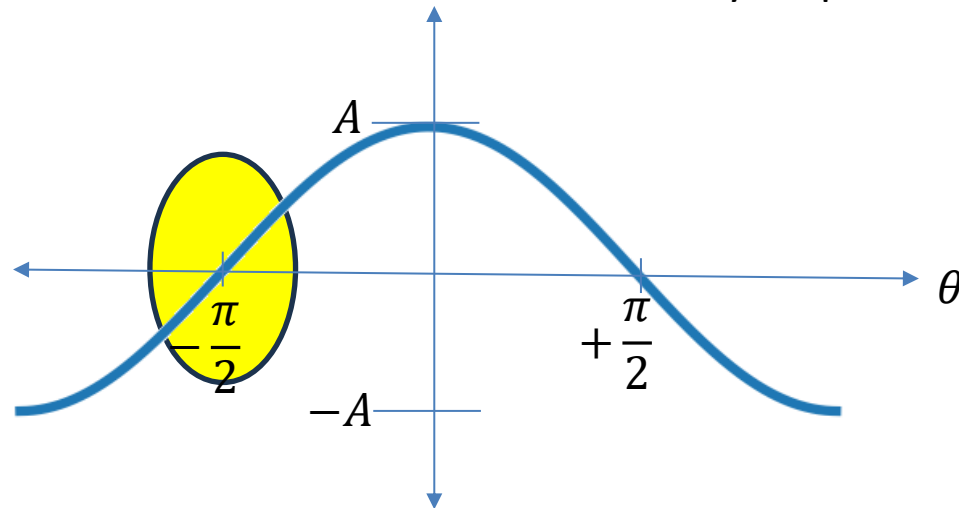
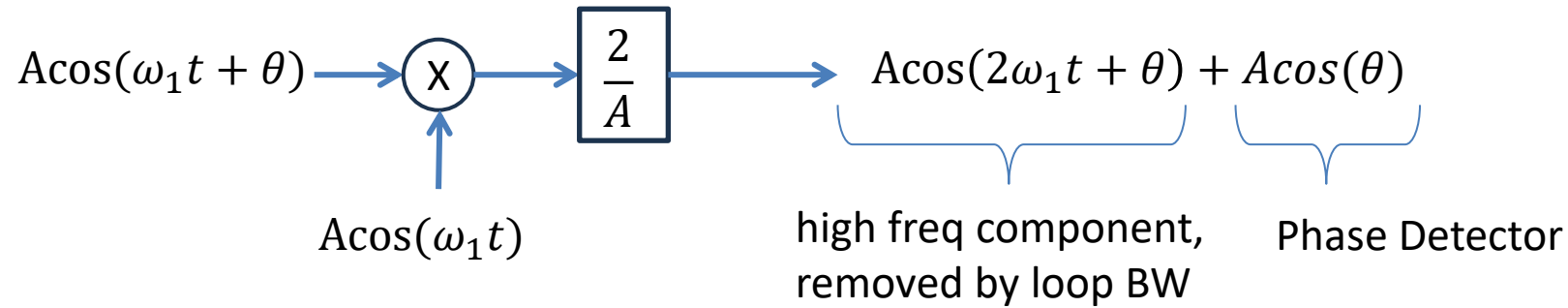


NCO Loop Model is

$$\frac{k_V}{z - 1} \text{ rads/count}$$

$$\text{Where } k_V = \frac{\pi}{2^{fcw_size}}$$

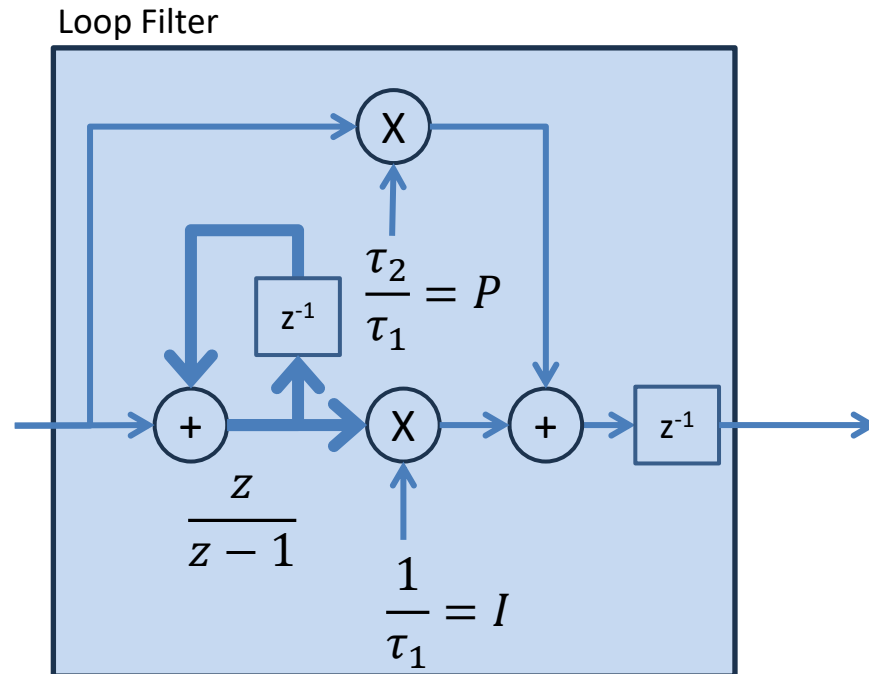
Multiplier as Phase Detector



At zero crossing where phase detector is linear with slope:

$$k_{PD} = A \text{ counts/rad}$$

Loop Filter

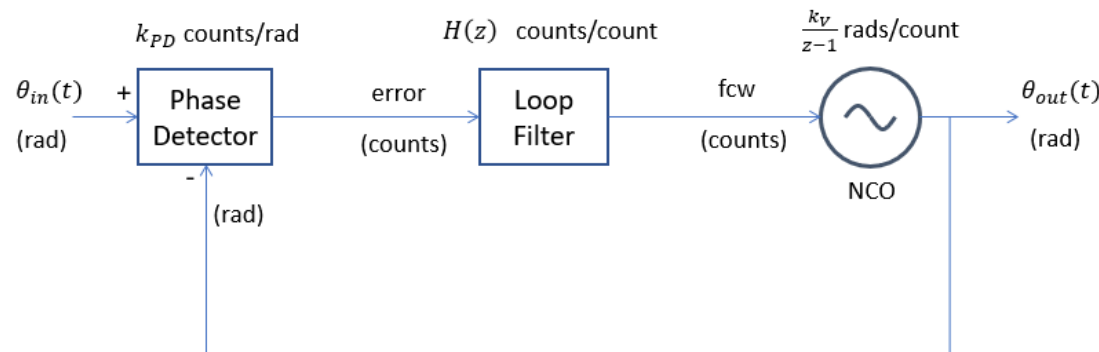


$$H(z) = P + I \frac{z}{z-1} (z^{-1})$$

$$= \frac{\tau_2}{\tau_1} + \frac{1}{\tau_1} \frac{1}{z-1}$$

$$H(z) = \frac{\tau_2 z + (1 - \tau_2)}{\tau_1 (z - 1)}$$

Open Loop Gain



$$G_{OL}(z) = \frac{k_V k_{PD}}{z-1} H(z)$$

$$= \frac{k_V k_{PD}}{z-1} \frac{(\tau_2 z + (1 - \tau_2))}{\tau_1 (z-1)}$$

$$= \frac{k_V k_{PD}}{\tau_1} \frac{(\tau_2 z + (1 - \tau_2))}{(z-1)^2}$$

See companion Marimo notebook `control_systems.py` for further simulation of this model



USING PYTHON FOR CONTROL SYSTEM SIMS

Python Control Systems Library



- <https://github.com/python-control/python-control>
- <https://python-control.readthedocs.io/en/>

The screenshot shows a web browser displaying the Python Control Systems Library documentation. The page has a dark blue header with the library name and version 0.10.2. A search bar is present. The left sidebar contains a 'USER GUIDE' with links to Introduction, Tutorial, Linear Systems, I/O Response and Plotting, Nonlinear Systems, Interconnected I/O Systems, Stochastic Systems, Examples, and Index. Below this is a 'REFERENCE MANUAL' section with links to Function Reference, Control System Classes, Package Configuration Parameters, MATLAB Compatibility Module, and Developer Notes. The main content area has the title 'Python Control Systems Library' and a description: 'The Python Control Systems Library (python-control) is a Python package that implements basic operations for analysis and design of feedback control systems.' It lists features such as linear and nonlinear system modeling, block diagram algebra, time response, frequency response, control analysis, control design, trajectory generation, model reduction, and estimator design. At the bottom, it provides links to the GitHub repository, issue tracker, and mailing list. A version dropdown menu shows 0.10.2 as the selected version.



Marimo Notebook

SIMULATION DEMONSTRATION

Simulation Files



GitHub public Repo

https://github.com/dsp-coach/grcon2026_control

- Complete installation guide for Python environment.
- Marimo notebook with simulation demonstration.



Scan for link to Github repository

Want More DSP?

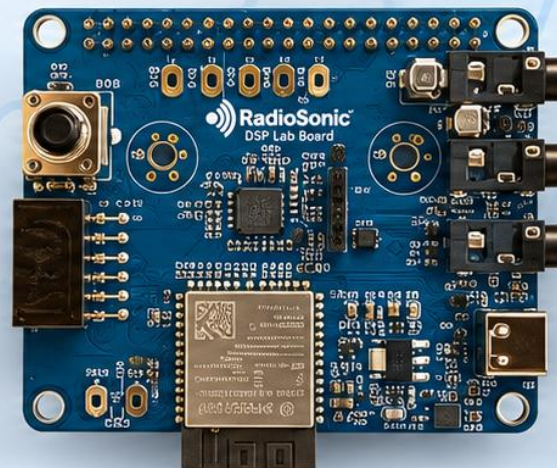


HANDS-ON DIGITAL FILTERS COURSE

~~\$349~~ **\$249**

with code **GRCON26**

RadioSonic hardware kit: \$49



SCAN TO REGISTER

Want More DSP?



Signal Processing for Software Radio

Course Kick-off: **October 1, 2026**

Early registration deadline: **September 24, 2026**

Late registrations allowed up to: **November 5, 2026**



Scan for more details



Python Applications for Digital Design and Signal Processing

Course Kick-off: **October 8, 2026**

Early registration deadline: **October 2, 2026**

Late registrations allowed up to: **November 5, 2026**

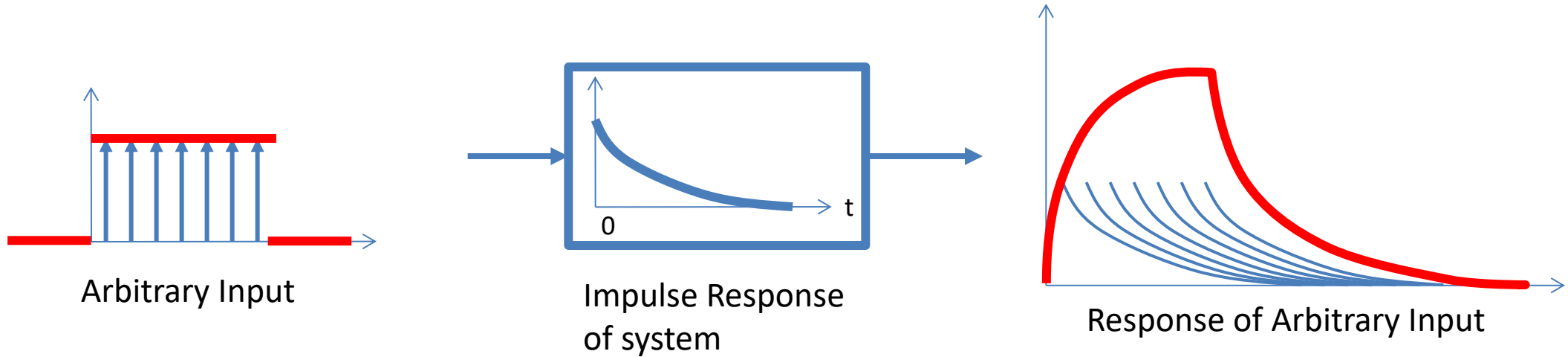


Thank you!



BACKUP SLIDES

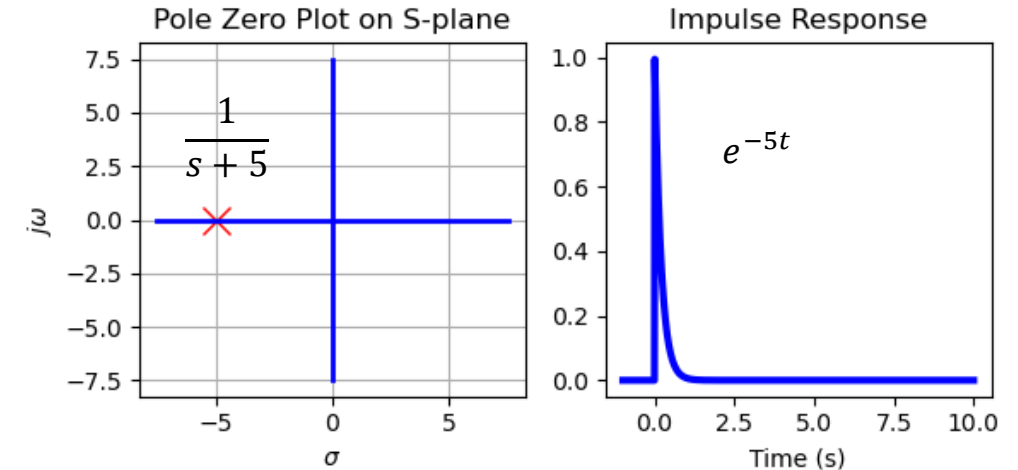
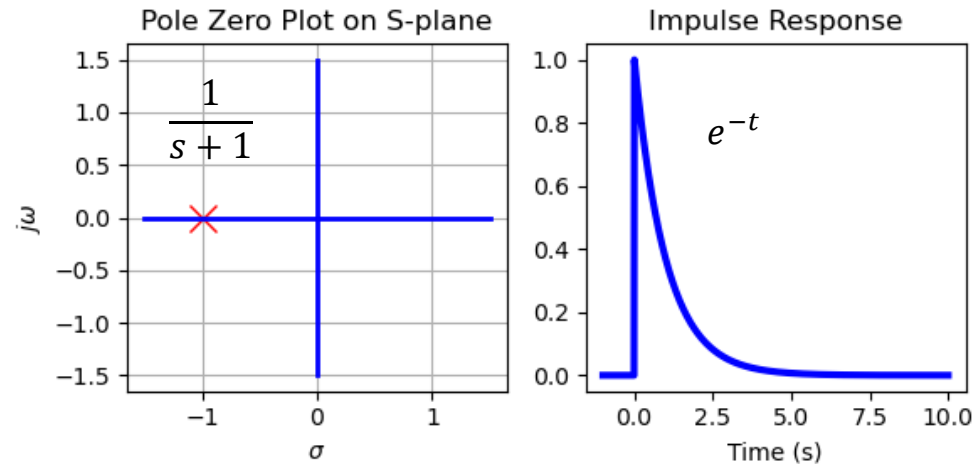
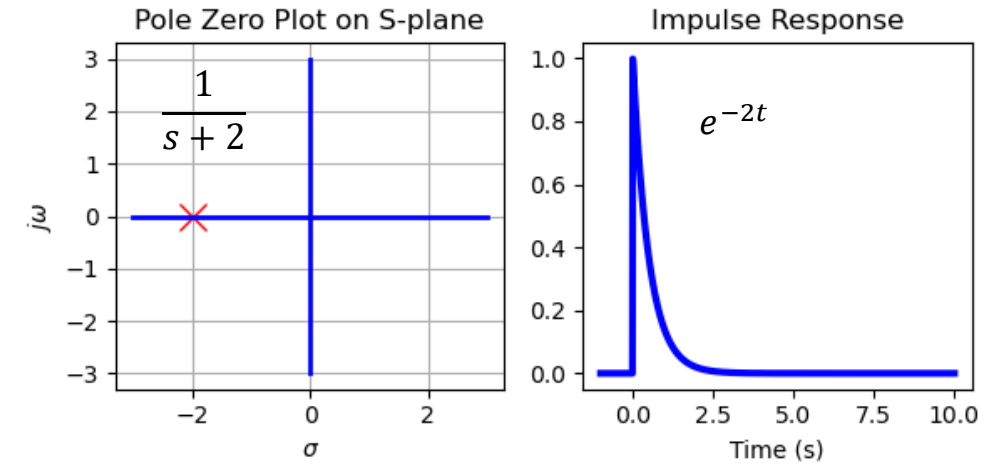
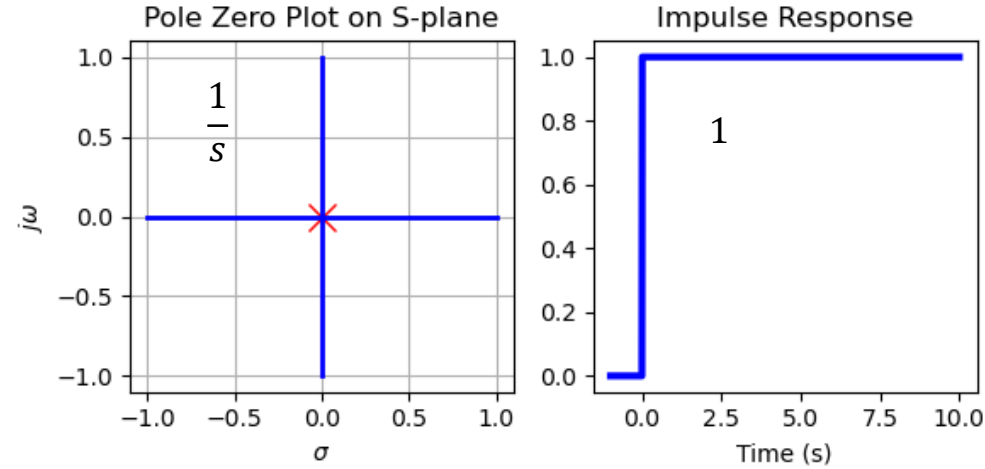
Laplace Transform



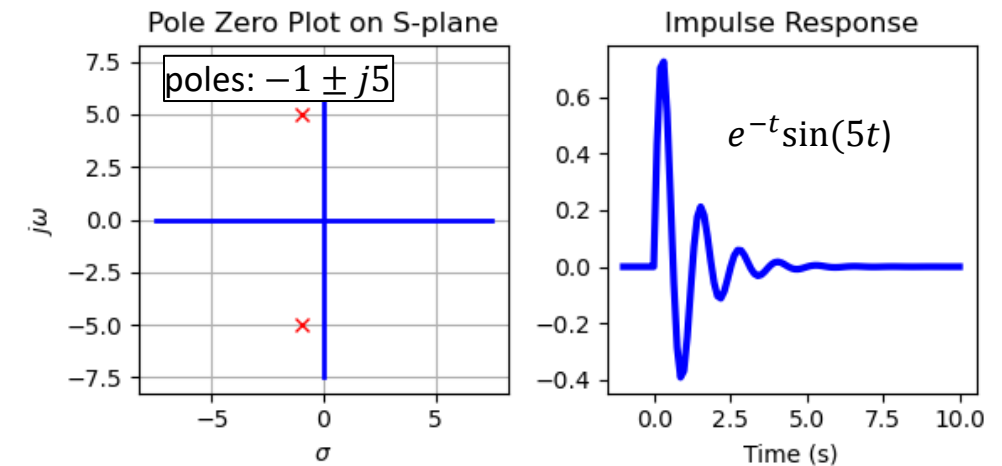
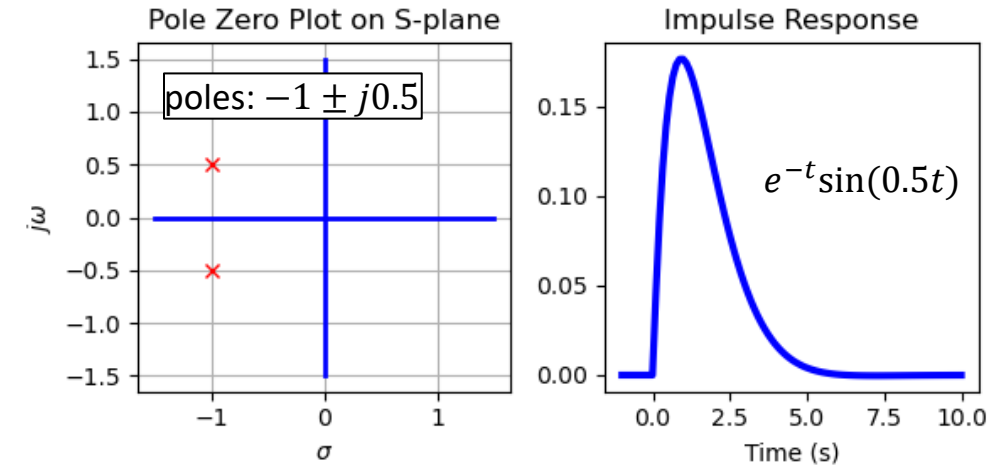
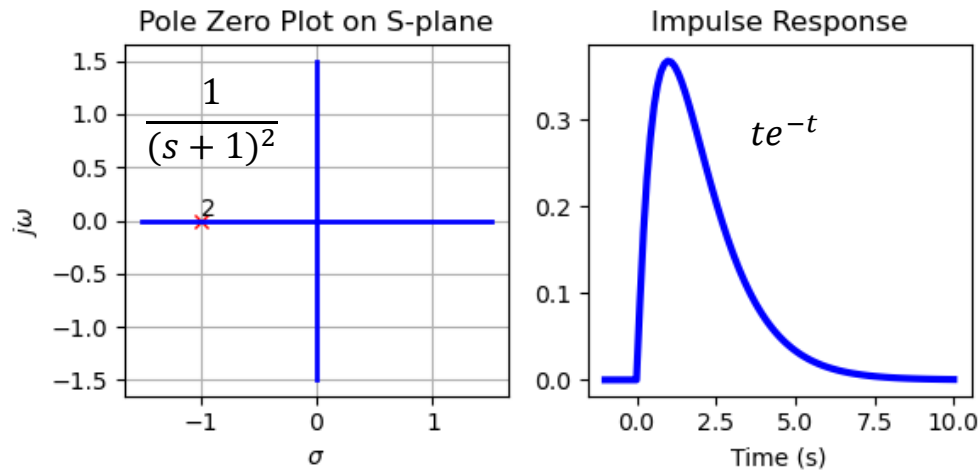
MOTIVATION FOR LAPLACE

- Convolution in the Time Domain is a product in the Frequency and Laplace Domains.
- In many cases it is easier for us to do a product rather than convolution.
- Can determine both steady state and transient responses
- The Laplace Transform of the system's impulse response provides useful behaviour insight.
- The Laplace Transform converts integro-differential equations to simple algebra.

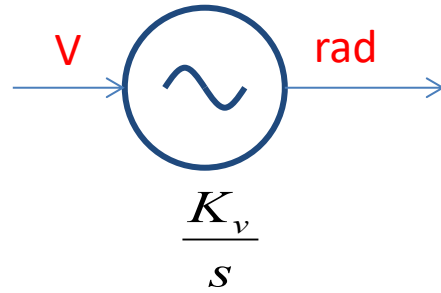
Laplace Transforms – First Order, All Pole



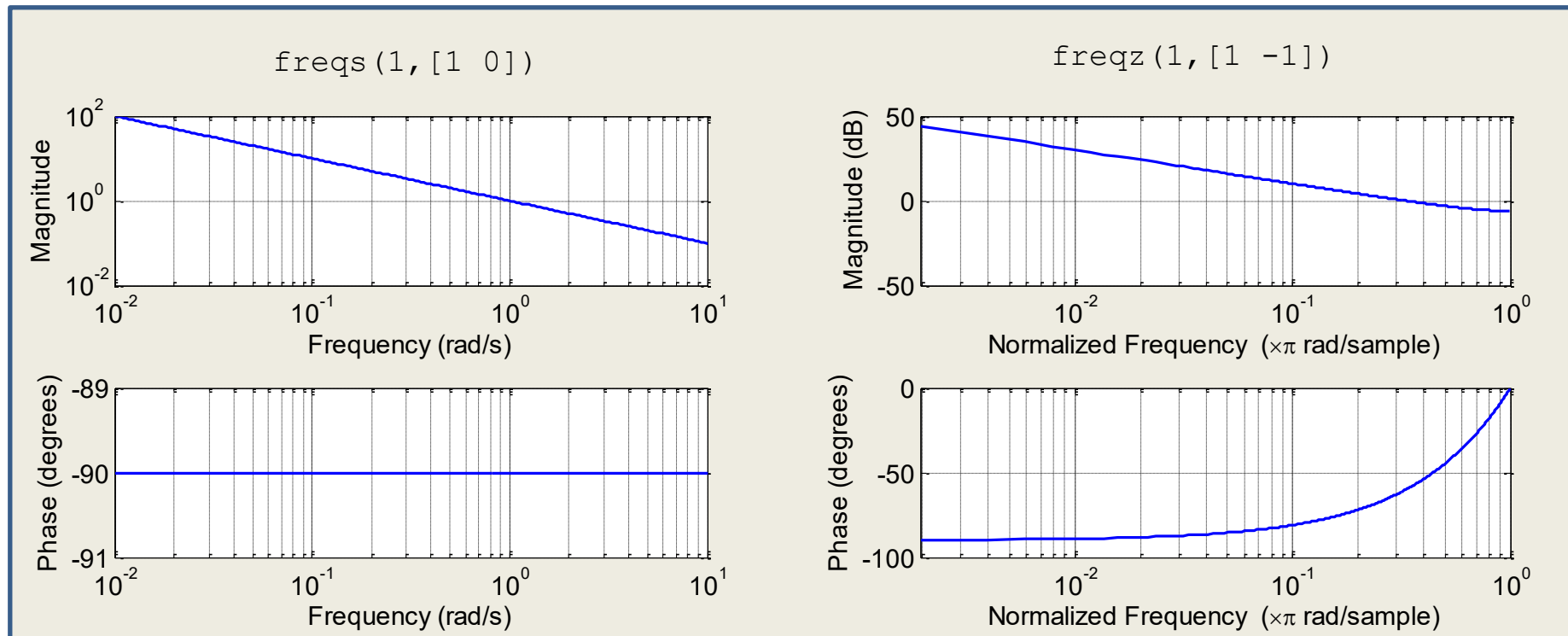
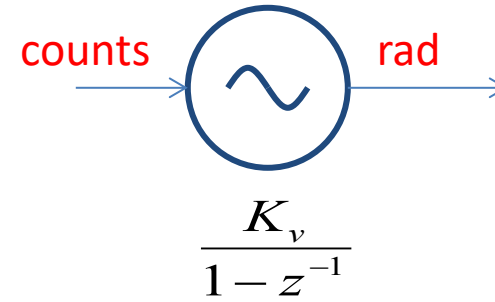
Example Laplace Transforms- 2nd Order, All Pole



VCO

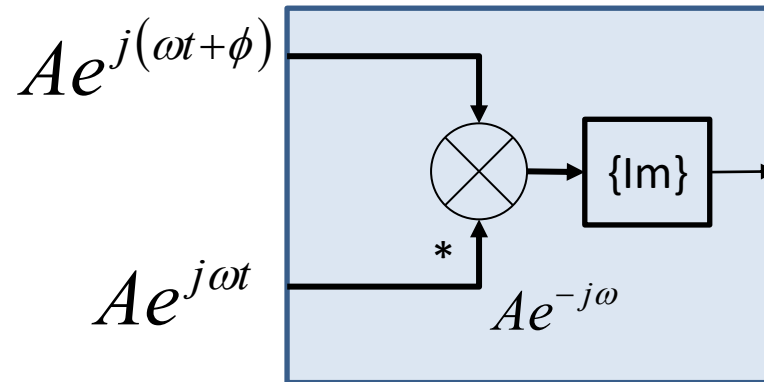


NCO



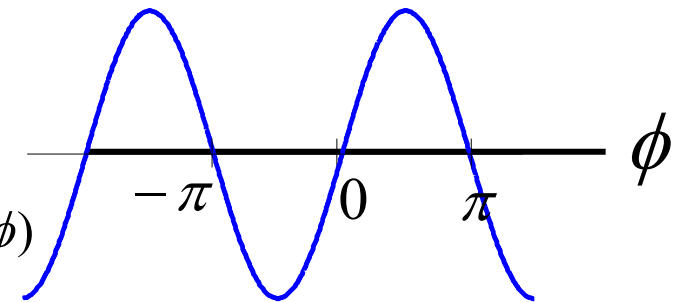
Phase Detectors

Complex Signals



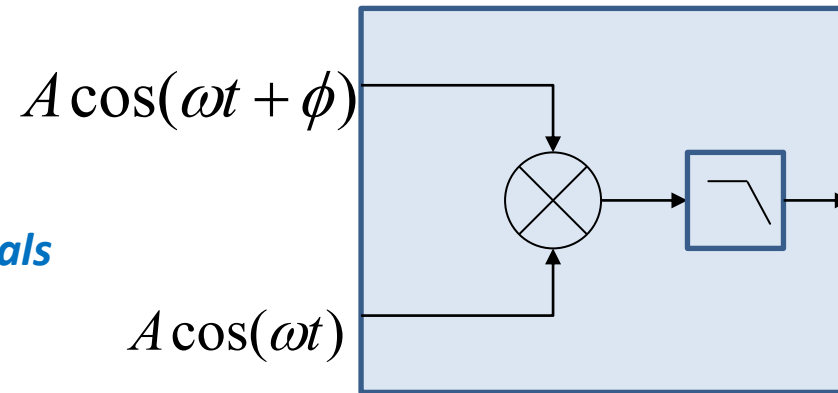
Phase error

$$A^2 \sin(\phi)$$



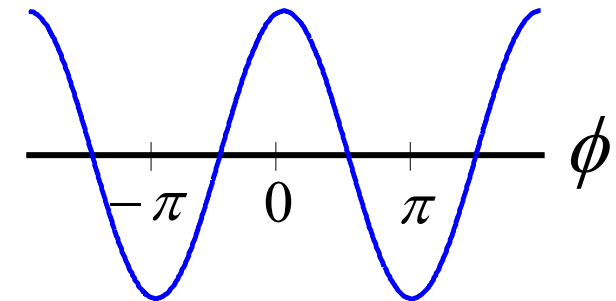
Or take Real output to get $\cos(\phi)$

Real Signals



Phase error

$$\frac{A^2}{2} \cos(\phi)$$



2nd Order System with PI Loop Filter



From classical continuous-time 2nd order loop equations with a PI loop filter:

τ_1 and τ_2 time constants determined from natural frequency and damping ratio:

ω_n : natural frequency rad/s

ζ : damping ratio

$$\tau_1 = \frac{K_v K_d}{\omega_n^2}$$
$$\tau_2 = \frac{2\zeta}{\omega_n}$$

And ω_n determined from desired loop BW, ω_{3dB} :

$$\omega_n = \frac{\omega_{3dB}}{\sqrt{\alpha + \sqrt{\alpha^2 + 1}}} \quad \alpha = 1 - 2\zeta^2$$

Applies when

$$G_{OL}(s) = \frac{K_v K_d (1 + s\tau_2)}{s^2 \tau_1}$$

From Closed Loop Canonical Form:

$$H_{CL}(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

Ref: Floyd M. Gardner,
Phaselock Techniques, John
Wiley and Sons, 1979

Loop Order and Type

$$G_{CL}(s) = \frac{G_F(s)}{1 + G_{OL}(s)}$$

Loop Order:

The degree of the characteristic equation

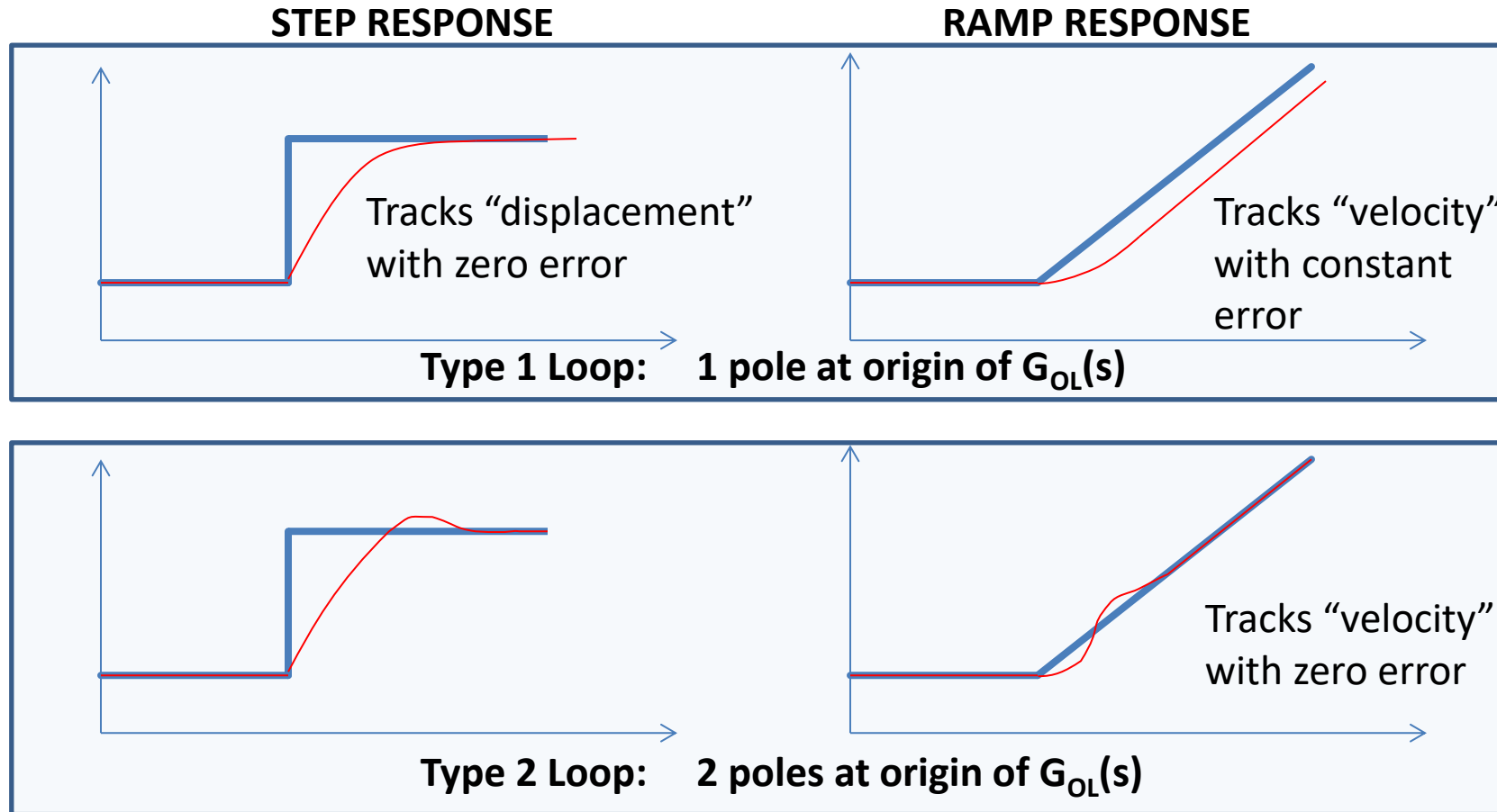
Loop Type:

The number of poles at the origin in $G_{OL}(s)$

Example:

2nd Order Type 1 Loop: $G_{OL}(s) = \frac{s + 3}{s(s + 2)}$

Loop Tracking



(Need a Type 3 Loop to track "acceleration")

Loop Tracking

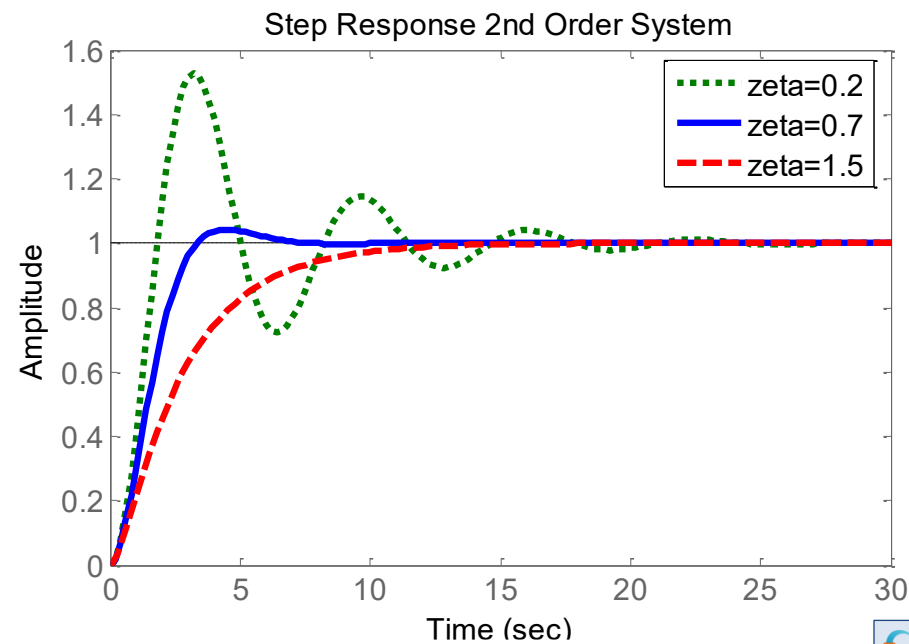


Final Value vs Input (Error) and Loop Type

Input	Type 0	Type 1	Type 2	Type 3
Step (Constant)	Constant	0	0	0
Velocity (Ramp)	Ramp	Constant	0	0
Acceleration (x^2)	x^2	Ramp	Constant	0

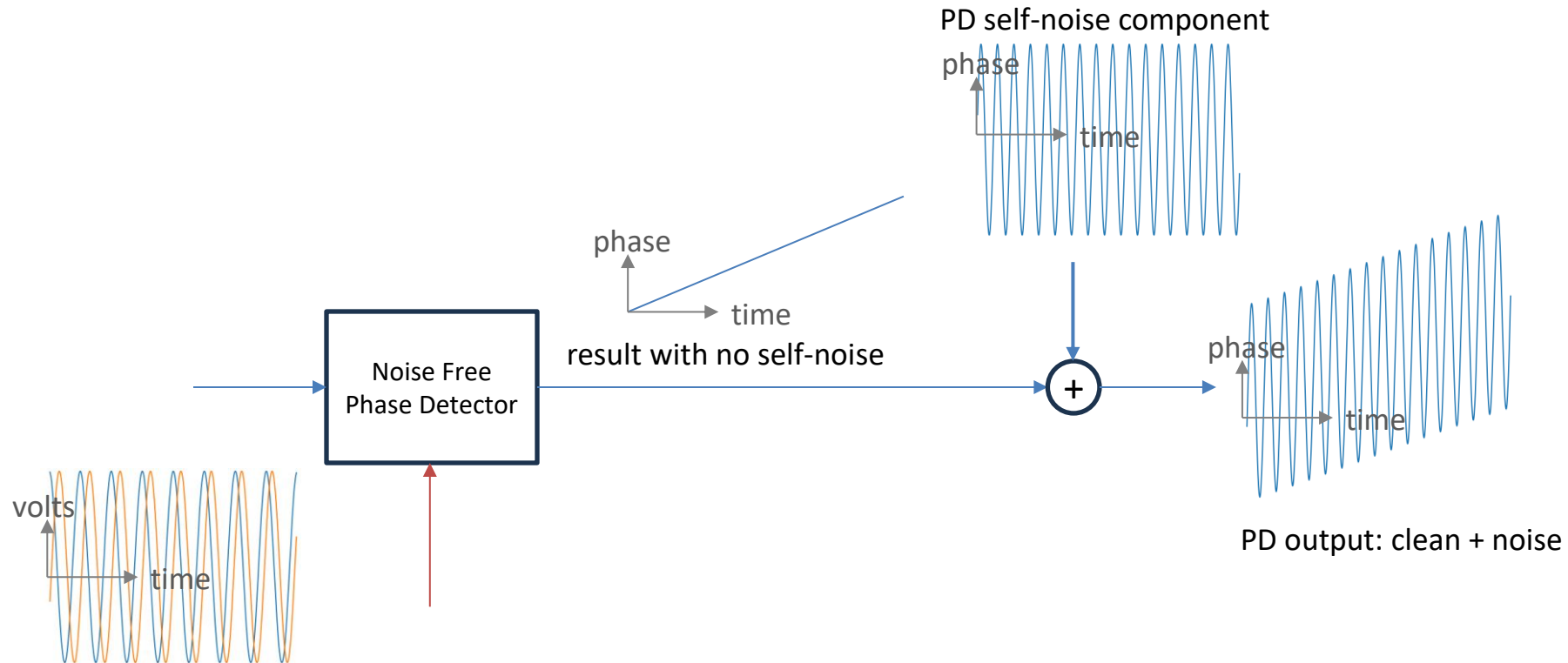
2nd Order System

Canonical Form: $H_{CL}(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$ ω_n : natural frequency
 ζ : damping ratio



```
sys=tf(1,[1 2*.707 1])  
step(sys)
```

Noisy Phase Detector is modelled in the loop as a noise-free phase detector with an external noise input



A MOTIVATION FOR USING USING $e^{j\phi}$: SSB Modulator

Consider a single tone at freq ω_1 $s_1(t) = e^{j\omega_1 t}$

We can shift the frequency by ω_Δ by multiplying (mixing) the signal with another single tone at freq ω_Δ , with no other filtering required after the product.

$$s_\Delta(t) = e^{-j\omega_\Delta t}$$

$$s_2(t) = s_1(t)s_\Delta(t) = e^{j\omega_1 t}e^{-j\omega_\Delta t} = e^{j(\omega_1 - \omega_\Delta)t} \quad \text{Done!}$$

Same processing but we will represent the signals with Sines and Cosines:

$$s_1(t) = \cos(\omega_1 t) + j\sin(\omega_1 t)$$

$$s_\Delta(t) = \cos(\omega_\Delta t) - j\sin(\omega_\Delta t)$$

$$s_2(t) = s_1(t)s_\Delta(t) = (\cos(\omega_1 t) + j\sin(\omega_1 t))(\cos(\omega_\Delta t) - j\sin(\omega_\Delta t))$$

$$= \cos(\omega_1 t)\cos(\omega_\Delta t) + \sin(\omega_1 t)\sin(\omega_\Delta t) + j(\sin(\omega_1 t)\cos(\omega_\Delta t) - \cos(\omega_1 t)\sin(\omega_\Delta t))$$

Ugh! Need I continue?

Motivation for using $e^{j\phi}$



De Moivre's Theorem:

$$(\cos(\theta) + j \sin(\theta))^n = \cos(n\theta) + j \sin(n\theta)$$

Same as:

$$(e^{j\theta})^n = e^{jn\theta}$$

BANNER IMAGE FOR NOTEBOOK



Control Systems

- GNU Radio Conference 2026
- Dan Boschen, The DSP Coach LLC

Control Cheat Sheet



`import control as con`

`sys = con.tf([num], [den])`
`sys = con.tf([num], [den], ts)`
`con.mineral(sys)`

transfer function of s: example `sys=con.tf(3, [1 2])` is $3/(s+2)$
transfer function of z, ts is sampling time interval
reduce transfer (good practice to always do this!)

`con.bode(sys)`
`con.nyquist(sys)`
`con.rlocus(sys)`

Bode plot (used with **open loop** tf to assess stability)
Nyquist plot (used with **open loop** tf to assess stability)
Root Locus plot (used with **open loop** tf to view closed loop poles vs gain)

`con.pzmap(sys)`
`con.step(sys)`
`con.impulse(sys)`

Map of poles and zeros
Step Response (used with **closed loop** tf)
Impulse Response (used with **closed loop** tf)

`[num, den] = con.zp2tf(z, p, k)`
`[z, p, k] = con.tf2zp([num], [den])`

zero-pole-gain to transfer function
transfer function to zero-pole-gain

`con.sensitivity`

sensitivity margin M for gain and phase margin for open loop TF
shortest distance on Nyquist plot to -1 is $1/M$
reasonable values for M: 1.3 to 2

`gcl = con.feedback(sys1,sys2)`

closed loop transfer function from forward tf and backward (feedback) loop tf